

4 Approximation in Value Space

Learning objectives

After studying this chapter, the reader should be able to:

1. connect the exact VI/PI and Q-factor equations of Chapter 2 to fitted and sample-based value-space methods, identify their error sources, and explain why repeated fitting can amplify Bellman errors;
2. derive Monte Carlo (MC), temporal-difference (TD), Q-learning, and SARSA updates under the book's cost-minimization convention and explain their on/off-policy and evaluation/optimality roles;
3. interpret replay, target networks, Double DQN, and dueling networks as modifications of approximate Q-value iteration, including the limits of tabular convergence claims under nonlinear approximation;
4. use an approximate value or base policy in rollout and multistep lookahead, interpret the corresponding performance bounds, and connect value-space approximation to policy-space methods; and
5. audit a learned value method using closed-loop cost, constraints, coverage, and computation rather than training loss alone.

Chapter roadmap

Chapter 2 introduced exact and approximate DP, and Chapter 3 developed the representation and fitting tools. This chapter brings them together in learning algorithms and online control. The first route is based on VI: fitted updates, Q-learning, and DQN construct targets from the Bellman optimality equation. The second is based on PI: we first develop the approximate-PI loop, then explain how MC and TD implement its evaluation block. SARSA couples sampled Q-factor evaluation to incremental policy improvement. We then examine how approximate values and base policies support online improvement through lookahead and rollout. A common mobility example compares the resulting methods, followed by an LP alternative and a chapter summary. Chapter 6 develops the connection to explicit policy approximation. Figure 4.1 shows the main progression.

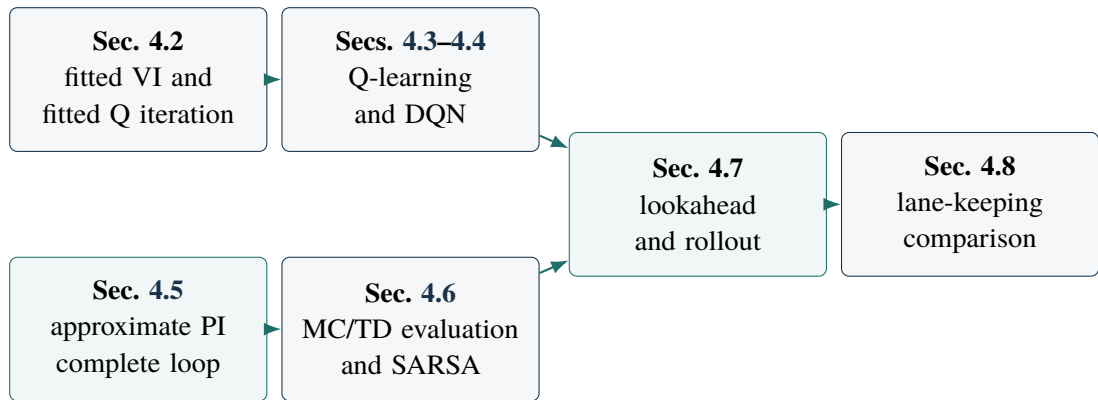


Figure 4.1: Chapter roadmap. The upper route develops VI-based methods; the lower introduces approximate PI before its sampled evaluation methods and SARSA. Arrows indicate conceptual progression, not iterations of an algorithm. Both routes connect to online lookahead and rollout.

4.1 From Approximate DP to Learning Algorithms

Chapter 2 identified where approximation enters DP. Chapter 3 explained how functions can be represented and fitted. This chapter develops the methods that generate those fitting targets and use the resulting approximations to construct and improve policies. The main reference is now a stationary infinite-horizon problem, so the same cost-to-go function can support successive decisions.

Three approximation sites in one decision.

Figure 4.2 recalls the value-space diagram from Chapter 2. For a discounted problem, its one-step policy construction is

$$\tilde{\mu}(x) \in \arg \min_{u \in \mathcal{U}(x)} \mathbb{E} [g(x, u, w) + \alpha \tilde{J}(f(x, u, w)) \mid x, u], \quad x \in \mathcal{X}. \quad (4.1)$$

Here $\tilde{J}$ is the approximate cost-to-go, f is the transition model, g is the stage cost, and $0 < \alpha < 1$ is the discount factor. The conditional expectation uses the disturbance law introduced in Chapter 2. This expression retains the same meaning for finite and continuous spaces; deterministic dynamics remove the expectation.

The three sites correspond to three computational questions. First, how is $\tilde{J}$ obtained and represented? Second, how is the expectation computed from a model or estimated from samples? Third, how is a feasible control selected by the minimization? A method can approximate more than one site. For example, a fitted cost-to-go can be combined with sampled successor predictions and a restricted search over candidate controls.

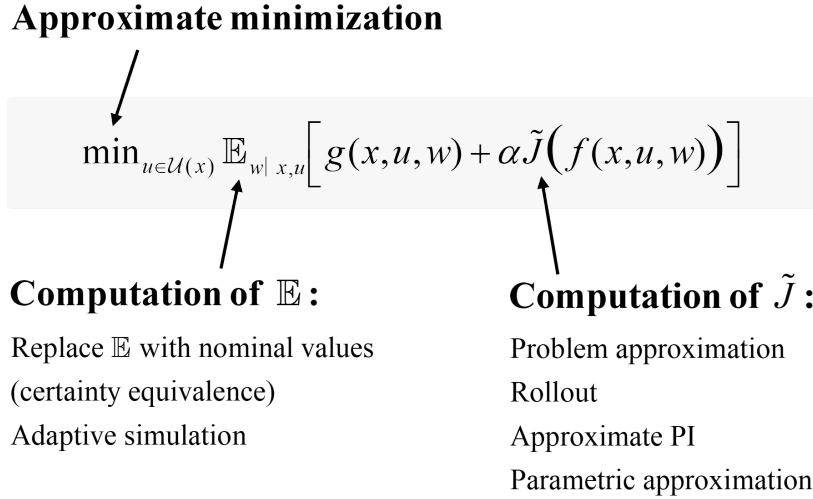


Figure 4.2: Three approximation sites in the policy construction of Equation (4.1): cost-to-go, expectation, and minimization. Repeated from Figure 2.13.

From a fitted function to a learning algorithm.

Chapter 3 supplied architectures such as $\tilde{J}(x; r)$ and procedures for fitting them to given targets. An architecture alone does not determine those targets. The same representation can approximate the optimal cost-to-go J^* or evaluate the cost-to-go J_μ of a specified policy. These lead to the two familiar DP routes:

- **Approximate VI** constructs new targets by applying a Bellman backup to the current approximation, then fits the next approximation. The reference is J^* , or Q^* when Q-factors are used. Fitted VI, Q-learning, and DQN are developed from this route.
- **Approximate PI** first approximates J_{μ_i} or Q_{μ_i} for the current policy, then uses that evaluation to improve the policy. MC and TD provide alternative ways to perform the evaluation; they do not improve the policy by themselves.

After the VI-based methods, Section 4.5 presents the complete approximate-PI loop. Section 4.6 then develops its MC/TD evaluation methods and SARSA's interleaving of evaluation and improvement. The essential distinction is what each target represents, not whether a table, fixed features, or a neural network stores the answer.

From offline approximation to online improvement.

A learned value can be used to construct a policy beforehand, or it can be retained for a new lookahead calculation at the current state during execution. A base policy can similarly supply a simulated cost-to-go for rollout. Thus offline approximation need not determine the final control without further computation: online search can improve the decision using the current state and the available model. The learned functions may remain fixed during this planning. Section 4.7 develops this connection between value-space learning and online policy improvement.

4.2 Fitted Value Iteration: Bellman Backup Followed by Projection

4.2.1 The generic fitted-VI step

Section 2.8 introduced the structural target–fit step of approximate VI. This section makes that step executable by specifying a function class, a training-state distribution, a projection or regression objective, and model- or sample-based Bellman targets.

For the stationary discounted problem introduced above, exact VI applies $J_{i+1} = TJ_i$ at every state. Suppose instead that values must lie in a class

$$\mathcal{F} = \{\tilde{J}(\cdot; \theta) : \theta \in \Theta\}. \quad (4.2)$$

Let Π_{ν_i} denote a fitting operation using a training-state distribution ν_i . A generic fitted or approximate VI step is

$$\tilde{J}_{i+1} = \Pi_{\nu_i} T\tilde{J}_i. \quad (4.3)$$

The notation is compact, but it represents two distinct computations.

First, construct a Bellman target. Its deterministic and stochastic forms are

$$y_i^{\text{det}}(x) = \min_{u \in \mathcal{U}(x)} \{g(x, u) + \alpha \tilde{J}_i(f(x, u))\}, \quad (4.4)$$

$$y_i^{\text{sto}}(x) = \min_{u \in \mathcal{U}(x)} \mathbb{E}[g(x, u, w) + \alpha \tilde{J}_i(f(x, u, w)) \mid x, u], \quad x \in \mathcal{X}. \quad (4.5)$$

Second, fit the target on samples $x^{(s)} \sim \nu_i$:

$$\theta_{i+1} \in \arg \min_{\theta} \sum_{s=1}^{q_i} \omega_i^{(s)} |\tilde{J}(x^{(s)}; \theta) - y_i(x^{(s)})|^2 + \lambda_i \mathcal{R}(\theta). \quad (4.6)$$

Here q_i is the number of training states, $\omega_i^{(s)} \geq 0$ are sample weights, and $\lambda_i \geq 0$ weights the regularizer $\mathcal{R}$. The old approximator remains fixed during target generation; the fitted parameters θ_{i+1} define $\tilde{J}_{i+1}$. The target depends on the previous approximator, so this is not a static supervised-learning problem. Approximation error can be regenerated at every iteration and can alter the greedy action used in the next target.

Equations (4.4)–(4.5) are *model-based target constructions*. The deterministic form requires the successor map f ; the stochastic form requires a conditional expectation of stage cost and successor value. Both also require minimizing over feasible actions at every sampled training state. A learned model or a simulator may replace an analytic model, but then model-estimation or Monte Carlo error must be added to the regression error. The sample-only alternatives that observe one successor without forming the expectation are developed in Sections 4.3 and 4.6.

The following algorithm makes the target–fit cycle explicit. The targets are frozen during each regression; they are regenerated only after the new approximator has been accepted.

Algorithm 4.1: Fitted value iteration

Input: Discount α , dynamics and cost model, feasible sets $\mathcal{U}(x)$, function class $\mathcal{F}$, initial parameters θ_0 , and iteration budget $I \geq 1$. For each iteration i , specify the sampling distribution ν_i , sample count q_i , nonnegative weights $\omega_i^{(s)}$ (not all zero), and regularization weight λ_i .

Initialize: Set $\tilde{J}_0(x) = \tilde{J}(x; \theta_0)$.

For $i = 0, \dots, I - 1$:

1. **Select training states.** Draw $x^{(s)} \sim \nu_i, s = 1, \dots, q_i$.
2. **Compute Bellman targets.** Hold θ_i fixed and, for every $s = 1, \dots, q_i$, compute

$$y_i^{(s)} = \min_{u \in \mathcal{U}(x^{(s)})} \mathbb{E}[g(x^{(s)}, u, w) + \alpha \tilde{J}(f(x^{(s)}, u, w); \theta_i) \mid x^{(s)}, u].$$

For deterministic dynamics, use Equation (4.4). A simulator-based implementation estimates the conditional expectation for each candidate control before minimizing.

3. **Fit the next approximator.** With all $y_i^{(s)}$ fixed, solve, or approximately solve with a declared fitting tolerance,

$$\theta_{i+1} \in \arg \min_{\theta \in \Theta} \left\{ \sum_{s=1}^{q_i} \omega_i^{(s)} [\tilde{J}(x^{(s)}; \theta) - y_i^{(s)}]^2 + \lambda_i \mathcal{R}(\theta) \right\}.$$

4. **Update.** Set $\tilde{J}_{i+1}(x) = \tilde{J}(x; \theta_{i+1})$.

Output: The fitted cost-to-go $\tilde{J}_I$. Its associated greedy policy is

$$\tilde{\mu}_I(x) \in \arg \min_{u \in \mathcal{U}(x)} \mathbb{E}[g(x, u, w) + \alpha \tilde{J}_I(f(x, u, w)) \mid x, u], \quad x \in \mathcal{X}.$$

The policy may be computed at queried states rather than stored everywhere.

The iteration budget specifies termination, not convergence. Check held-out target error, Bellman residual, and induced-policy performance separately. A small change between successive fits does not imply a small Bellman residual: the projected iteration may have reached a fixed point of $\Pi_\nu T$ rather than of T .

4.2.2 Approximate Q-value iteration

The Q-factor version uses the operator introduced with exact VI in Section 2.7. In expectation form,

$$(T_Q Q)(x, u) = \mathbb{E} \left[g(x, u, w) + \alpha \min_{v \in \mathcal{U}(x^+)} Q(x^+, v) \mid x, u \right], \quad x^+ = f(x, u, w). \quad (4.7)$$

Fitted Q iteration is

$$\tilde{Q}_{i+1} = \Pi_{Q, \rho_i} T_Q \tilde{Q}_i, \quad (4.8)$$

where ρ_i is a state–control sampling distribution. With a finite action set, the stored approximation immediately yields

$$\tilde{\mu}_i(x) \in \arg \min_{u \in \mathcal{U}(x)} \tilde{Q}_i(x, u). \quad (4.9)$$

This execution advantage explains why Q-factor approximation is prominent in model-free RL. It does not remove the action minimization for a continuous control set; Chapter 6 introduces a separate actor for that case.

4.2.3 How one-step approximation errors accumulate

The next proposition separates Bellman contraction from implementation error. It follows by perturbing the discounted VI contraction, as in the approximate-VI analysis of Bertsekas (2019, Sec. 4.4) and Bertsekas (2025, Sec. 3.3.1). It is not a convergence guarantee for an arbitrary regression algorithm.

Assumption 4.1 (Discounted setting for the error bounds). Let $0 < \alpha < 1$, let stage costs be uniformly bounded, and consider bounded measurable cost-to-go functions. Assume the expectations are well defined, the Bellman operators preserve this function class, and the stated minima admit measurable admissible minimizing policies.

These conditions give the sup-norm contraction used below, with $\|J\|_\infty = \sup_{x \in \mathcal{X}} |J(x)|$. They include the finite-state, finite-control setting of assumption 2.3, but do not require discretization of a continuous state or control space. Unbounded-state LQR with quadratic costs is not covered by this bounded-cost assumption without additional analysis. Finite tables and matrices below are explicitly identified as implementations for finite models.

Proposition 4.1 (Perturbed value-iteration bound). *Suppose*

$$\tilde{J}_{i+1} = T\tilde{J}_i + e_i \quad \text{and} \quad \|e_i\|_\infty \leq \varepsilon_i \quad (4.10)$$

under assumption 4.1. Then

$$\|\tilde{J}_i - J^*\|_\infty \leq \alpha^i \|\tilde{J}_0 - J^*\|_\infty + \sum_{j=0}^{i-1} \alpha^{i-1-j} \varepsilon_j. \quad (4.11)$$

If $\varepsilon_j \leq \bar{\varepsilon}$, the second term is at most $\bar{\varepsilon}(1 - \alpha^i)/(1 - \alpha)$.

Proof. The contraction property gives

$$\|\tilde{J}_{i+1} - J^*\|_\infty \leq \alpha \|\tilde{J}_i - J^*\|_\infty + \varepsilon_i.$$

Recursive substitution proves the stated bound. □

The bound is conditional: one constant $\bar{\epsilon}$ must bound the backup error over *all states and all iterations*, as already emphasized in Equation (2.92). Least-squares fitting controls a distribution-weighted error at the current iteration instead. A small training loss therefore does not establish this uniform bound: poorly sampled states can have large errors, and successive targets can magnify the fitting residual. Moreover, bounded error only keeps the iterates within an error neighborhood; it does not imply that they converge to a single function. Thus the error-bound analysis is useful, but a practical fitted-VI implementation still needs a separate stability and convergence argument. The following example shows why even exact least-squares fitting can fail to keep the iterates bounded.

Value error and policy error are different.

If the uniform backup error is at most $\bar{\epsilon}$ at every iteration, the proposition gives $\limsup_i \|\tilde{J}_i - J^*\|_\infty \leq \bar{\epsilon}/(1 - \alpha)$. For policies greedy with respect to $\tilde{J}_i$, the lookahead bound proved in proposition 4.3 also gives

$$\limsup_i \|J_{\tilde{\mu}_i} - J^*\|_\infty \leq \frac{2\alpha\bar{\epsilon}}{(1 - \alpha)^2} \leq \frac{2\bar{\epsilon}}{(1 - \alpha)^2}.$$

The last expression is the convenient bound reported in Bertsekas (2019, Sec. 4.4). Neither statement says that the iterates converge. More importantly, a small regression loss at one iteration does not establish the required uniform backup-error bound.

Worked Example 4.1 (Exact fitting can amplify Bellman errors). Consider the two-state system in Figure 4.3. There is one available control at each state: state 1 moves to state 2, and state 2 stays there. Every transition costs zero. Thus there is no policy-selection or sampling difficulty to obscure the effect of fitting. For $0 < \alpha < 1$, the Bellman equations and exact VI are

$$J^*(1) = \alpha J^*(2), \quad J^*(2) = \alpha J^*(2), \quad J^* = (0, 0),$$

$$J_{i+1}(1) = \alpha J_i(2), \quad J_{i+1}(2) = \alpha J_i(2).$$

Exact VI converges to zero from any finite initial values.

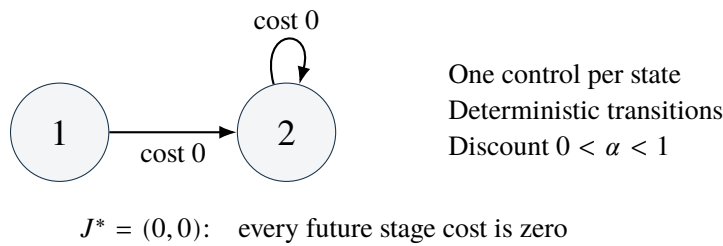


Figure 4.3: Two-state discounted problem for worked example 4.1. State 2 has a zero-cost self-loop; there is no choice of control. The example isolates error caused by repeated approximation, not by exploration or inaccurate model information. Redrawn from Bertsekas (2019, Fig. 4.4.1).

Now restrict the approximator to $\tilde{J}_i = (r_i, 2r_i)$: one scalar parameter determines both values. This architecture can represent the solution exactly, by choosing $r_i = 0$. Each fitted iteration nevertheless has two distinct steps:

1. **Compute the Bellman target.** Since both successors are state 2,

$$T\tilde{J}_i = (\alpha\tilde{J}_i(2), \alpha\tilde{J}_i(2)) = (2\alpha r_i, 2\alpha r_i).$$

This target generally does not lie on the line $(r, 2r)$.

2. **Fit the target within the architecture.** With positive state weights ξ_1, ξ_2 , choose

$$r_{i+1} = \arg \min_r \{ \xi_1 (r - 2\alpha r_i)^2 + \xi_2 (2r - 2\alpha r_i)^2 \}.$$

The weights specify how strongly each state's target is fitted; equal weights give the states equal importance.

Setting the derivative of this quadratic objective to zero gives

$$2\xi_1(r_{i+1} - 2\alpha r_i) + 4\xi_2(2r_{i+1} - 2\alpha r_i) = 0, \quad r_{i+1} = \alpha \frac{2(\xi_1 + 2\xi_2)}{\xi_1 + 4\xi_2} r_i.$$

For equal weights the multiplier is $6\alpha/5$, so any $5/6 < \alpha < 1$ produces divergence from a nonzero initialization. In particular, $\alpha = 0.9$ gives $r_{i+1} = 1.08r_i$: $r_0 = 1$ leads to $r_1 = 1.08$ and $r_{10} \approx 2.159$. The model, Bellman targets, and least-squares minimizations are all exact; it is their repeated combination with the restricted architecture that amplifies error.

Figure 4.4 illustrates these two steps geometrically. For $r_i = 1$, the current pair $(1, 2)$ produces the target $(1.8, 1.8)$. Its sup-norm error decreases from 2 to 1.8 under the Bellman step. The equal-weight least-squares projection then gives $(1.08, 2.16)$, whose error is 2.16, greater than the original 2. In general the fitting residual is

$$e_i = (1.08r_i, 2.16r_i) - (1.8r_i, 1.8r_i) = (-0.72r_i, 0.36r_i).$$

Thus $\|e_i\|_\infty = 0.72|r_i|$ grows without bound for nonzero r_0 . The example does not contradict proposition 4.1: it violates the uniform-error condition needed for a bounded asymptotic error. Adapted from Bertsekas (2019, Example 4.4.1).

In Figure 4.4, the coordinate axes are the costs at the two states, not physical state coordinates. The navy line contains every representable pair $(r, 2r)$; the dashed diagonal contains pairs with equal costs at both states. The Bellman target lies on that diagonal because both states have the same successor. Equal-weight fitting is orthogonal projection in these cost coordinates, whereas changing the weights changes the projection direction and the point selected on the navy line. For this target, the possible fitted parameters range between 0.9 and 1.8: emphasis on state 2 approaches $2r = 1.8$, while emphasis on state 1 approaches $r = 1.8$.

The fitting weights therefore affect stability, not just prediction accuracy. Sampling more frequently from states visited by a useful policy is a practical design choice, but is not a general

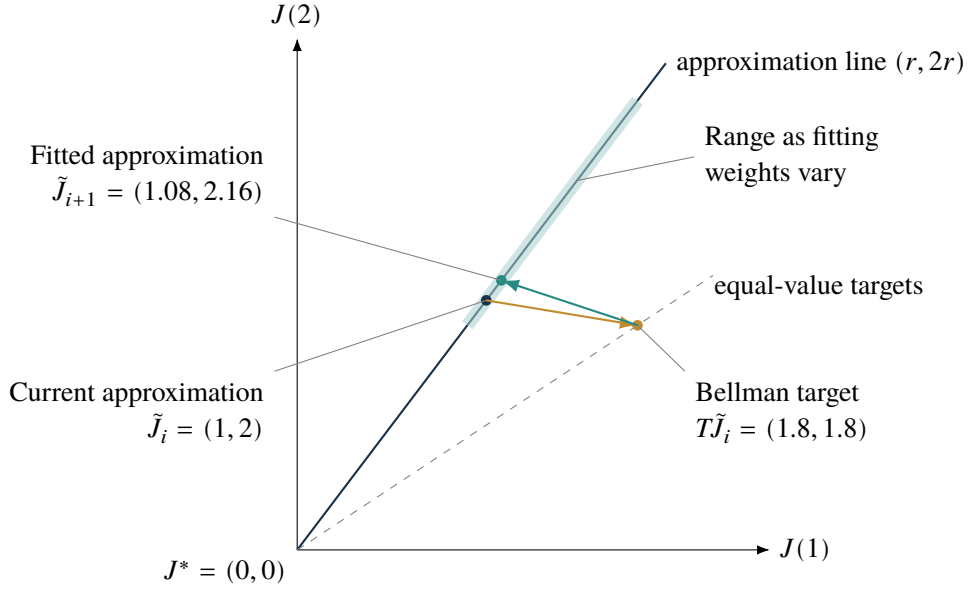


Figure 4.4: One fitted-VI update for $\alpha = 0.9$ and $r_i = 1$. Gold arrow: compute the exact Bellman target. Teal arrow: project onto the approximation line using equal fitting weights. The resulting point is farther from J^* than the current approximation. The pale segment indicates possible projections under other positive fitting weights (endpoints are limiting cases). Redrawn from Bertsekas (2019, Fig. 4.4.2).

convergence theorem for fitted VI. In this example, stability requires $\alpha 2(\xi_1 + 2\xi_2)/(\xi_1 + 4\xi_2) < 1$. This illustrates why Chapter 3’s fixed-target least-squares solution alone cannot settle the stability of repeated fitting.

4.3 Q-Learning as Sampled Q-Value Iteration

The fitted Q iteration in Section 4.2 updates a function from Bellman targets. Q-learning instead updates visited entries incrementally from individual transitions. Q-factors absorb the one-step model comparison into a state–control function. Chapter 2 showed that one transition target is an unbiased sample of a Q-Bellman backup component. Here that structural bridge is turned into an incremental learning algorithm. Given a sampled transition (x_k, u_k, g_k, x_{k+1}) , tabular Q-learning applies

$$y_k^Q = g_k + \alpha \min_{v \in \mathcal{U}(x_{k+1})} Q_k(x_{k+1}, v), \quad (4.12)$$

$$Q_{k+1}(x_k, u_k) = Q_k(x_k, u_k) + \eta_k [y_k^Q - Q_k(x_k, u_k)], \quad (4.13)$$

and leaves all unvisited entries unchanged. Here k counts sampled learning steps and η_k is the stepsize for the visited entry. For a fixed current table and a correctly sampled transition,

$$\mathbb{E}[y_k^Q \mid Q_k, x_k, u_k] = (T_Q Q_k)(x_k, u_k).$$

The left side averages over the sampled cost and successor state. A single y_k^Q generally differs from that expectation; equality without the expectation holds for deterministic transitions and costs. Q-learning is therefore an asynchronous, sampled version of Q-value iteration (Watkins and Dayan 1992).

The update corrects a prediction using the observed one-step cost and the current prediction of what follows. It does not wait to observe the entire future return. Using a current estimate inside the target is called *bootstrapping*; the difference between this target and the present prediction is a *temporal-difference (TD) error*:

$$\delta_k^Q = g_k + \alpha \min_{v \in \mathcal{U}(x_{k+1})} Q_k(x_{k+1}, v) - Q_k(x_k, u_k), \quad (4.14)$$

and Equation (4.13) is simply $Q_{k+1}(x_k, u_k) = Q_k(x_k, u_k) + \eta_k \delta_k^Q$. The word TD is not reserved for fixed-policy state-value evaluation. Q-learning is a TD *control* method because the minimum in its target also selects a greedy continuation, rather than evaluating a specified continuation policy. Section 4.6 later develops TD evaluation for a fixed policy; the target-minus-prediction idea is the same.

Q-learning is *off-policy*: the behavior policy selects exploratory controls to collect data, whereas the target uses a greedy continuation. The next behavior action need not minimize the current Q-factor. Off-policy therefore describes the relation between the behavior and target policies, not the absence of a data-generating policy. This separation lets the same optimality target use data from different exploratory policies, provided the required state–control pairs are sufficiently covered. Section 4.6.5 later contrasts this target with SARSA’s on-policy evaluation target.

AI/RL terminology bridge

Q-learning is called *model free* because its update needs a sampled next state and cost rather than an explicit transition matrix. This does not imply that no model influenced the simulator, feature design, data collection, or safety filter. Conversely, model-based fitted Q iteration and model-free Q-learning can use the same Q-network architecture; they differ in how the Bellman target is evaluated.

From one sampled update to the algorithm.

The TD error δ_k^Q measures target minus current prediction. In contrast, target minus its conditional expectation is sampling noise: it has zero conditional mean but need not vanish for a single transition. Q-learning uses successive TD errors to move visited entries toward their Bellman backups without a complete sweep over all pairs at each step. The following algorithm assembles action selection, data collection, and this one-entry update into a repeated learning procedure.

Algorithm 4.2: Tabular Q-learning

Input: Finite state and feasible-control sets, discount $0 < \alpha < 1$, access to sampled system transitions and costs, an exploratory behavior policy, a pair-specific stepsize schedule, and update budget K .

Initialize: Choose finite values $Q_0(x, u)$ for every feasible state–control pair and an initial nonterminal state x_0 .

For $k = 0, \dots, K - 1$:

1. **Collect a transition.** Select $u_k \in \mathcal{U}(x_k)$ from the behavior policy, apply it, and observe cost g_k and successor state x_{k+1} .
2. **Compute the target.** Set

$$y_k^Q = g_k + \alpha \min_{v \in \mathcal{U}(x_{k+1})} Q_k(x_{k+1}, v).$$

At a genuine zero-cost terminal state, set $y_k^Q = g_k$ instead. The minimum is independent of the next behavior action.

3. **Update the visited entry.** Using the stepsize η_k assigned to this pair’s current visit, set

$$Q_{k+1}(x_k, u_k) = Q_k(x_k, u_k) + \eta_k[y_k^Q - Q_k(x_k, u_k)].$$

Set $Q_{k+1}(x, u) = Q_k(x, u)$ for every other feasible pair.

4. **Continue.** Use the observed successor as the next current state. If the episode terminated, reset to a new initial state instead.

Output: The table Q_K and its greedy policy

$$\tilde{\mu}_K(x) \in \arg \min_{u \in \mathcal{U}(x)} Q_K(x, u) \quad \text{for every nonterminal state } x.$$

What tabular convergence does and does not say.

For a stationary finite discounted MDP with bounded one-stage costs, tabular Q-learning converges under standard stochastic-approximation conditions when every state–control pair is visited infinitely often and its stepsizes obey

$$\sum_{k \in \mathcal{H}(x, u)} \eta_k = \infty, \quad \sum_{k \in \mathcal{H}(x, u)} \eta_k^2 < \infty. \quad (4.15)$$

Here $\mathcal{H}(x, u) = \{k : (x_k, u_k) = (x, u)\}$ contains the learning-iteration indices at which this particular pair is updated, not physical time values. For example, a pair visited at steps 2, 7, 11 receives stepsizes $\eta_2, \eta_7, \eta_{11}$; updates to other entries do not train this entry. The conditions must therefore hold along each pair’s own visits. Equivalently, one may index a pair’s stepsizes by its local visit count $n = 1, 2, \dots$ and apply the usual infinite-sum conditions to that sequence.

The statement does not transfer automatically to nonlinear function approximation, where a parameter update can affect many state–control estimates and off-policy bootstrapping can be

unstable. The combination of function approximation, bootstrapping, and off-policy learning is often called the *deadly triad*. It is a diagnostic warning, not a theorem that every such method must diverge.

From a table to a parametric Q-factor.

For a differentiable approximator, the sampled semi-gradient update is

$$\theta_{k+1} = \theta_k + \eta_k [y_k - \tilde{Q}(x_k, u_k; \theta_k)] \nabla_{\theta} \tilde{Q}(x_k, u_k; \theta_k).$$

Here $y_k = y_k^Q$ is the greedy target. A table stores a separate scalar for each state–control pair, so one update changes only that entry. A parametric approximator instead uses the same parameter vector θ to represent Q-values at many pairs. Updating those parameters using one transition can therefore also change predictions at other pairs. For example, in $\tilde{Q}(x, u; \theta) = \phi(x, u)^{\top} \theta$, a coefficient contributes at every pair where its corresponding feature is nonzero; a neural network similarly reuses its weights across inputs. Thus tabular convergence cannot be transferred just by replacing the table with a neural network.

Two practical difficulties become important when training this approximator from consecutive transitions. First, successive states along a trajectory are often similar, so successive training samples are correlated. Updating immediately from each one can repeatedly emphasize a narrow part of the state space. Second, the same parameters determine both the current prediction and the successor Q-values in y_k^Q . Although the target is held fixed within one gradient update, recomputing it after that update changes the label being fitted, even for the same stored transition. The next section introduces DQN’s responses to these two issues: reuse randomly selected past transitions and separate the target-generating parameters from those being updated.

4.4 Deep Q-Networks as Stabilized Fitted Q Iteration

Following Q-learning, a deep Q-network (DQN) replaces a table by $\tilde{Q}(x, u; \theta)$ while retaining a finite action set. The two difficulties at the end of Section 4.3 motivate two distinct design choices. *Experience replay* reuses randomly selected past transitions to reduce the dominance of consecutive, correlated samples. A separate *target network* supplies bootstrap labels that do not change after every update of the fitted network.

To implement experience replay, store observed transitions (x, u, g, x') in a *replay buffer* $\mathcal{D}$, the collection of past experience. For each training update, draw a random *minibatch*, a small subset of transitions from $\mathcal{D}$, rather than using only the most recent transition once. These names describe a method, its storage, and one sampled training batch, respectively; they are not different replay methods. Reusing observations supports several learning updates per interaction, and random sampling reduces the dominance of temporally adjacent observations. It does not make the underlying data independent or guarantee coverage of unseen actions.

For each sampled transition, the *online network* with parameters θ predicts the cost of the observed pair (x, u) . A second network with parameters θ^- evaluates possible controls at the

successor x' to construct the bootstrap target. The basic squared target loss is

$$y^{\text{DQN}} = g + \alpha \min_{v \in \mathcal{U}(x')} \tilde{Q}(x', v; \theta^-), \quad (4.16)$$

$$\mathcal{L}(\theta) = \mathbb{E}_{(x, u, g, x') \sim \mathcal{D}} \left[(\tilde{Q}(x, u; \theta) - y^{\text{DQN}})^2 \right]. \quad (4.17)$$

DQN retains Q-learning's off-policy target: a replayed transition can come from an earlier exploratory policy, while the target minimizes the target network's Q-factor. In Figure 4.5, the two networks reach the loss for different reasons: the online network supplies the *prediction being fitted*, and the target network supplies the *label used to fit it*. During a gradient step, y^{DQN} is held constant. Gradients update θ , not θ^- .

Original DQN periodically copies θ into θ^- and holds the target network fixed between copies (Mnih et al. 2015). This lets several updates fit a temporarily fixed target function instead of chasing a label that moves after every gradient step. A common alternative is a slow copy $\theta^- \leftarrow (1 - \tau)\theta^- + \tau\theta$ with small $0 < \tau < 1$. Copying too frequently weakens the separation; copying too infrequently leaves stale targets. Replay and delayed targets address the preceding instability mechanisms, but do not restore a general contraction or convergence guarantee for neural Q-learning.

Two complementary variants modify this construction: Double DQN changes how the target is computed, whereas Dueling DQN changes how Q-values are represented.

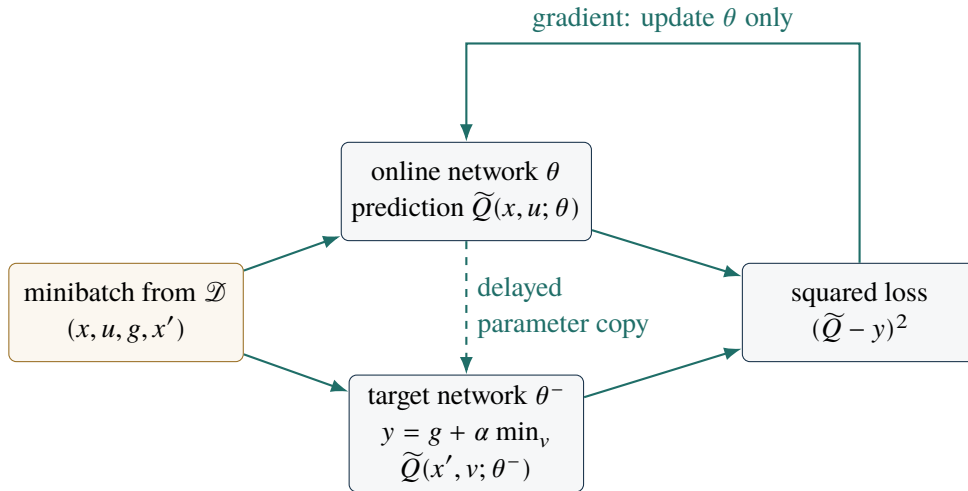


Figure 4.5: DQN training from stored transitions. Compared with updating a single network from consecutive fresh transitions, the replay buffer $\mathcal{D}$ supplies random minibatches and a delayed network supplies the bootstrap labels. The online prediction and fixed label enter the same loss, but only the online parameters receive its gradient. The dashed arrow copies parameters; it is not a gradient update.

Double DQN: separate action selection from evaluation.

In reward-maximizing notation, using the same estimates to select and evaluate a maximum can create positive maximization bias. Under the present cost-minimization convention, the analogous reuse can create a downward minimization bias. Even if each action estimate is individually unbiased, the smallest estimate is preferentially one whose error is negative. For example, let two true action costs both be 10, with independent estimation errors equal to -1 or $+1$ with equal probability. The minimum estimated cost is 9 in three of the four equally likely cases and 11 in the fourth, so its expectation is 9.5, not 10. Selecting the minimum and reporting that same estimate introduces bias. Double DQN reduces this reuse by separating selection and evaluation:

$$v^* \in \arg \min_{v \in \mathcal{U}(x')} \tilde{Q}(x', v; \theta), \quad (4.18)$$

$$y^{\text{Double}} = g + \alpha \tilde{Q}(x', v^*; \theta^-). \quad (4.19)$$

The online network selects the action and the target network evaluates it (Hasselt et al. 2016).

The two lines have deliberately different jobs. In Equation (4.18), the online parameters answer the discrete decision question “which continuation action currently looks best?” In Equation (4.19), the delayed parameters attach a numerical cost to that already selected action. Basic DQN instead uses $\min_{v \in \mathcal{U}(x')} \tilde{Q}(x', v; \theta^-)$, so the same noisy target-network outputs both choose and score the minimum. Separation reduces, but does not eliminate, selection bias. It also does not create two independently trained optimal Q-functions; the target network remains a delayed copy.

For example, if the online outputs at x' are (4, 3, 5) and the target outputs are (2, 6, 7), Double DQN selects the second action and evaluates it as 6, whereas basic DQN uses the target-network minimum 2 from the first action. The discrepancy is not an algebra error: it is exactly the selection–evaluation separation. Under cost minimization, noisy minima tend to be too low; under the reward-maximizing convention, noisy maxima tend to be too high. Figure 4.6 shows how this changes the target branch of the DQN diagram. It does not change the prediction being fitted or the squared-loss training rule.

Dueling DQN: separate shared cost from action differences.

Dueling DQN denotes DQN with a *dueling network architecture*, the architecture introduced by Wang et al. (2016). Unlike Double DQN, it changes the representation of Q, not the Bellman target. Its motivation is that action values at a state share a common cost level, while control selection depends on their differences. When several actions have similar consequences, learning the shared part can benefit all of them rather than treating each action value separately. This is a useful structural bias, not a guarantee of improved performance.

The network therefore produces a scalar $V(x)$ and an action-dependent output $A(x, u)$ from shared features. The latter is called the *advantage stream* because it represents relative action differences; its precise relation to the advantage function is discussed below. Simply setting $\tilde{Q}(x, u) = V(x) + A(x, u)$ leaves their roles ambiguous: adding any state-dependent scalar $c(x)$ to $V(x)$ and subtracting it from every $A(x, u)$ leaves Q unchanged. For example, the same Q pair

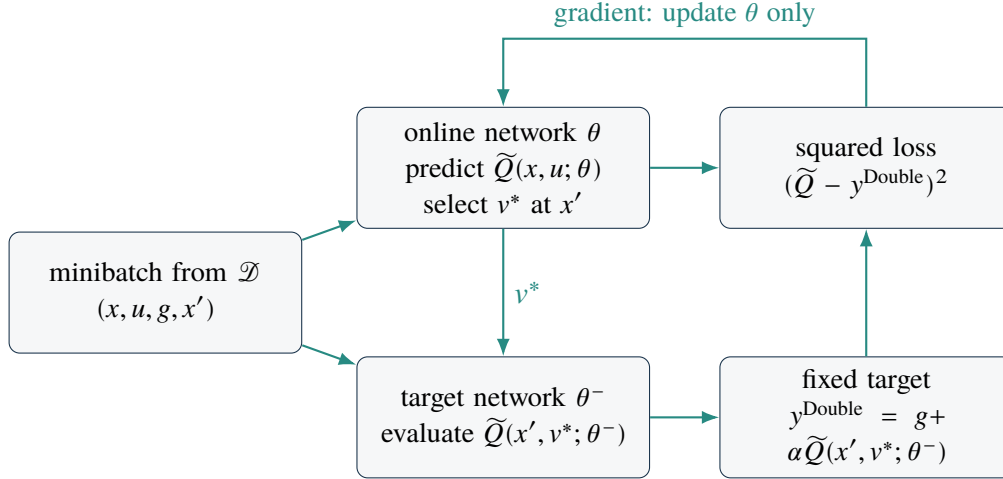


Figure 4.6: Double DQN in the same prediction–target–loss structure as Figure 4.5. The online network selects v^* ; the target network evaluates that selected action instead of taking its own minimum. Delayed parameter copying is unchanged and omitted here for clarity. No gradient passes through target construction or action selection.

$(10, 12)$ can be represented by $V = 10, A = (0, 2)$ or by $V = 11, A = (-1, 1)$. Training against Q targets alone cannot distinguish these decompositions.

To assign the common level to V , the usual dueling combination subtracts the mean of the raw action outputs:

$$\tilde{Q}(x, u) = V(x) + A(x, u) - \frac{1}{|\mathcal{U}|} \sum_{v \in \mathcal{U}} A(x, v). \quad (4.20)$$

Here the finite action set $\mathcal{U}$ is common to all states. Write $\bar{A}(x) = |\mathcal{U}|^{-1} \sum_{v \in \mathcal{U}} A(x, v)$. Averaging Equation (4.20) over actions gives

$$V(x) = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \tilde{Q}(x, u), \quad A(x, u) - \bar{A}(x) = \tilde{Q}(x, u) - V(x).$$

Thus V carries the mean action cost and the centered output carries deviations from it. For $(10, 12)$ these quantities are 11 and $(-1, 1)$. Subtracting the same quantity from every action preserves their ordering: $\arg \min_{u \in \mathcal{U}} \tilde{Q}(x, u) = \arg \min_{u \in \mathcal{U}} A(x, u)$. The decomposition into V and the *centered* output is now fixed by Q. The raw output A still admits an arbitrary common shift; centering does not make every network output or parameter uniquely identifiable.

Both streams are trained *jointly* through the combined Q prediction, not by separate supervised fits to known V and A labels. For one replayed pair (x, u) , let $d = \tilde{Q}(x, u) - y^{\text{DQN}}$ and $M = |\mathcal{U}|$. Holding the target fixed, the shared loss d^2 has output derivatives

$$\frac{\partial d^2}{\partial V(x)} = 2d, \quad \frac{\partial d^2}{\partial A(x, v)} = 2d \left(\mathbf{1}_{\{v=u\}} - \frac{1}{M} \right), \quad v \in \mathcal{U}.$$

Here the indicator is one when $v = u$ and zero otherwise. The chain rule then carries these derivatives through each stream’s parameters and the shared feature layers, exactly as in Chapter 3.

Thus one Q-target trains both branches; it does not supply two separate target values. Figure 4.7 shows this computation. Mean centering is an architectural choice, not a consequence of the definition of advantage. Subtracting $\min_v A(x, v)$ is another choice under cost minimization. The original reward-maximizing paper compares max subtraction with mean subtraction and adopts the latter for smoother optimization behavior: changes to the selected extremal action do not alone determine the centering term (Wang et al. 2016, Sec. 3, Eqs. (8)–(9)).

Relation to the advantage function.

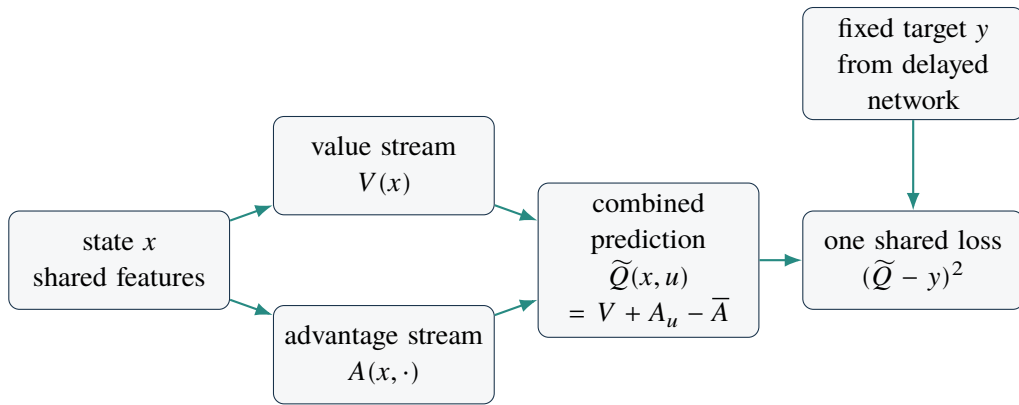
The name of the advantage stream is motivated by the usual decomposition of an action cost into a policy’s cost and a relative action difference. To make that connection precise under cost minimization, define the *advantage function* for policy μ by

$$A_\mu(x, u) = Q_\mu(x, u) - J_\mu(x). \quad (4.21)$$

The subscript μ specifies the reference policy. The function measures the excess cost of taking u once and then following μ , relative to following μ immediately. For a deterministic policy, $A_\mu(x, \mu(x)) = 0$; a negative value identifies a one-step improvement over the current policy. For the optimal functions,

$$A^*(x, u) = Q^*(x, u) - J^*(x) \geq 0, \quad \min_{u \in \mathcal{U}(x)} A^*(x, u) = 0. \quad (4.22)$$

Only differences across actions affect the greedy decision. This motivates approximating Q differences or advantages when a large state-dependent offset is harder to learn than the action ordering.



Backpropagation through the combined prediction updates both streams and shared features; the target branch is held fixed.

Figure 4.7: Dueling DQN expands the online-network box in Figure 4.5 into two jointly trained streams. Replay supplies the selected control u and the target-transition data. The target network has the same dueling architecture with delayed parameters. The loss can also use a Double DQN target.

The dueling outputs should therefore not be identified directly with these policy-specific functions. With mean centering, $V(x)$ is the mean of the predicted action costs, not generally $J_\mu(x)$ or $J^*(x) = \min_u Q^*(x, u)$. The centered output has zero *uniform action mean*, not necessarily zero value at $\mu(x)$ or zero minimum. Dueling DQN and advantage updating share an interest in action differences, but are not identical algorithms.

Double DQN and Dueling DQN are also complementary, not competing alternatives: the former changes target construction, while the latter changes the Q-network architecture. A dueling network can be trained with the Double DQN target, as in Wang et al. (2016).

Control viewpoint

DQN is natural for a finite maneuver or mode library: lane change left, keep lane, lane change right; charge, hold, discharge; or a finite set of motion primitives. A steering wheel or motor torque is physically continuous. A DQN controller for that variable either discretizes it or selects a finite primitive. Keeping a continuous action requires an inner minimization of a structured Q-function or a policy-space actor such as those developed in Chapter 6.

4.5 Approximate Policy Iteration

Exact PI in Section 2.7 alternated an exact solution of $J_{\mu_i} = T_{\mu_i} J_{\mu_i}$ and an exact greedy improvement, while Section 2.8 introduced the structural evaluation–improvement loop of approximate PI. Here that loop is instantiated with fitted and sample-based evaluators and its errors are quantified. The approximate form is needed because solving for all policy values is expensive in a large or unknown system and a newly improved policy may not have a compact representation:

$$\text{evaluation: } \tilde{J}_{\mu_i} \approx J_{\mu_i} \quad \text{or} \quad \tilde{Q}_{\mu_i} \approx Q_{\mu_i}, \quad (4.23)$$

$$\text{improvement: } \mu_{i+1}(x) \in \arg \min_{u \in \mathcal{U}(x)} \mathbb{E}[g(x, u, w) + \alpha \tilde{J}_{\mu_i}(f(x, u, w)) \mid x, u], \quad x \in \mathcal{X}, \quad (4.24)$$

or, with a Q-factor critic,

$$\mu_{i+1}(x) \in \arg \min_{u \in \mathcal{U}(x)} \tilde{Q}_{\mu_i}(x, u). \quad (4.25)$$

An *evaluation target* is a training label that estimates the cost of the current policy from a sampled state or state–control pair. It is not the optimal cost unless that policy is already optimal. Targets may be computed from model equations or from data: a *return* is a sum of discounted trajectory costs, with a terminal estimate for any omitted remainder. MC evaluation uses these returns, while sampled TD updates combine an observed cost and successor with a current cost-to-go estimate. The latter bootstrap target is refined as evaluation proceeds. If the improved policy is too large to store, actions from Equation (4.24) can become labels for a parameterized actor. That last regression is approximation in *policy space*; it is developed in Chapter 6.

MC and TD are alternative implementations of this evaluation block, not competing controllers. Their update rules, including the linear least-squares TD (LSTD) method, follow in Section 4.6. First, the complete sample-based approximate PI loop is

1. collect trajectories under μ_i or a declared behavior policy;
2. fit $\tilde{J}_{\mu_i}$ or $\tilde{Q}_{\mu_i}$ using MC, TD, LSTD, or another evaluator;
3. form an improved policy from a model-based J minimization, a direct Q minimization, or a fitted actor;
4. independently audit the candidate and either accept it or collect more data.

Model use can therefore differ between evaluation and improvement. For example, TD evaluation is sample based and model free, while the improvement in Equation (4.24) explicitly uses a transition model. With a Q-factor critic, evaluation can use sampled transitions and improvement can compare the stored Q estimates directly. The latter needs no explicit successor model, but is a minimization, not itself a sampled evaluation.

Two implementations of the same loop.

Figure 4.8 contrasts two implementations: cost-function evaluation and Q-factor evaluation. For a fixed current policy μ_i , generate sample trajectories indexed by s . A cost-function target starts at $x_0^{(s)} = x^{(s)}$ and uses μ_i at every simulated stage:

$$\beta_J^{(s)} = \sum_{t=0}^{N_s-1} \alpha^t g(x_t^{(s)}, \mu_i(x_t^{(s)}), w_t^{(s)}) + \alpha^{N_s} \hat{J}_T(x_{N_s}^{(s)}), \quad (4.26)$$

$$r_{J,i} \in \arg \min_r \sum_s [\tilde{J}_{\mu_i}(x^{(s)}; r) - \beta_J^{(s)}]^2. \quad (4.27)$$

Here N_s is the simulated length and $\hat{J}_T$ approximates the remaining cost of μ_i . The resulting J critic is used in the model-based improvement Equation (4.24). Evaluation may use sampled trajectories even though improvement requires a model-based expectation or a simulator that can compare candidate controls.

For the Q-factor variant, choose an initial pair $(x^{(s)}, u^{(s)})$, apply $u^{(s)}$ once, and follow μ_i thereafter:

$$\beta_Q^{(s)} = g(x^{(s)}, u^{(s)}, w_0^{(s)}) + \sum_{t=1}^{N_s-1} \alpha^t g(x_t^{(s)}, \mu_i(x_t^{(s)}), w_t^{(s)}) + \alpha^{N_s} \hat{J}_T(x_{N_s}^{(s)}), \quad (4.28)$$

$$r_{Q,i} \in \arg \min_r \sum_s [\tilde{Q}_{\mu_i}(x^{(s)}, u^{(s)}; r) - \beta_Q^{(s)}]^2. \quad (4.29)$$

The trailing stages evaluate the current policy, not an optimal policy. Improvement then minimizes $\tilde{Q}_{\mu_i}(x, u; r_{Q,i})$ over $u \in \mathcal{U}(x)$ as in Equation (4.25). Neither this sampled evaluation nor direct Q minimization needs an explicit transition model. This is the model-free variant in the backup sense of Chapter 1; using a simulator to obtain data is a separate model-use question.

Improvement and policy storage.

The minimization can be exact or approximate. Its selected controls can be stored explicitly in a finite model, recomputed when needed, or used as targets for a policy approximator from

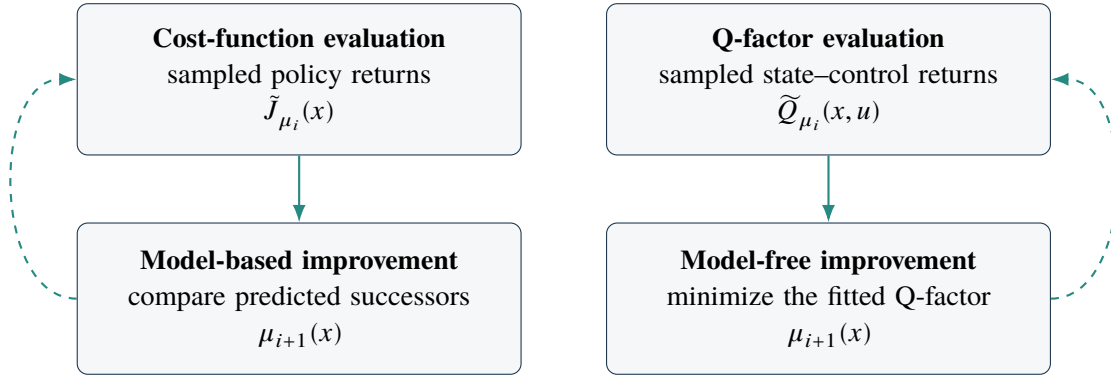


Figure 4.8: Two approximate-PI implementations. Both may evaluate by simulation; the distinction is whether improvement predicts successors or directly compares Q estimates. Dashed arrows start the next policy iteration.

Chapter 3. For continuous controls, a typical actor fit is

$$r_{\mu, i+1} \in \arg \min_r \sum_s \|\tilde{\mu}(x^{(s)}; r) - u_{\text{improved}}^{(s)}\|^2.$$

For finite controls, the corresponding problem is policy classification. An actor fit adds policy-space approximation error; it is optional, not part of the definition of a value critic. Chapter 6 develops this additional step.

Exploration when reusing trajectories.

After the first control, a deterministic-policy trajectory contains only pairs $(x, \mu_i(x))$. Reusing its tails therefore evaluates the policy's chosen actions but does not evaluate untried alternatives. Deliberately varied initial state-control pairs or appropriately corrected exploratory data are needed for improvement. Another construction first fits $\tilde{J}_{\mu_i}$ from trajectories following μ_i . It then chooses exploratory pairs $(x^{(s)}, u^{(s)})$, where $u^{(s)}$ need not equal $\mu_i(x^{(s)})$. Apply that chosen control once and record its cost $g^{(s)} = g(x^{(s)}, u^{(s)}, w^{(s)})$ and successor $x^{+(s)} = f(x^{(s)}, u^{(s)}, w^{(s)})$. A Q-regression target is

$$y^{(s)} = g^{(s)} + \alpha \tilde{J}_{\mu_i}(x^{+(s)}).$$

The first term charges the *exploratory control*; the second estimates the cost of following μ_i thereafter. Thus this target evaluates an alternative first action without requiring a new full trajectory for each alternative. Fit $\tilde{Q}_{\mu_i}(x, u)$ to these labels and compare actions using the resulting Q-function. This separates state-value learning from action coverage, at the price of two approximation stages (Bertsekas 2019, Sec. 4.7.3).

4.5.1 Evaluation error, improvement error, and the approximate PI error region

Two different inaccuracies must not be collapsed into one training loss. Assume a uniform evaluation error

$$\|\tilde{J}_{\mu_i} - J_{\mu_i}\|_{\infty} \leq \delta \quad (4.30)$$

and an approximate-greedy error

$$T_{\mu_{i+1}} \tilde{J}_{\mu_i} \leq T \tilde{J}_{\mu_i} + \epsilon \mathbf{1}. \quad (4.31)$$

The inequality holds at every state. Since T minimizes over admissible controls, its left-hand side is also no smaller than $T \tilde{J}_{\mu_i}$. Thus the condition is equivalently $\|T_{\mu_{i+1}} \tilde{J}_{\mu_i} - T \tilde{J}_{\mu_i}\|_\infty \leq \epsilon$. Exact minimization gives $\epsilon = 0$ and equality between the two Bellman expressions, even if the evaluator itself is inaccurate. A restricted maneuver search, an incompletely optimized continuous action, or a fitted actor can make $\epsilon > 0$ even with a perfect critic.

Proposition 4.2 (Discounted approximate PI performance region). *Under assumption 4.1, suppose Equations (4.30) and (4.31) hold at every approximate PI iteration. Then the generated policies satisfy*

$$\limsup_{i \rightarrow \infty} \|J_{\mu_i} - J^*\|_\infty \leq \frac{\epsilon + 2\alpha\delta}{(1-\alpha)^2}. \quad (4.32)$$

If the policy sequence eventually becomes constant, with policy $\bar{\mu}$, the stronger bound

$$\|J_{\bar{\mu}} - J^*\|_\infty \leq \frac{\epsilon + 2\alpha\delta}{1-\alpha} \quad (4.33)$$

holds.

Proof. Set $c = \epsilon + 2\alpha\delta$. Comparing the approximate and exact policy values in the improvement step gives $T_{\mu_{i+1}} J_{\mu_i} \leq T J_{\mu_i} + c \mathbf{1}$. Since $T J_{\mu_i} \leq J_{\mu_i}$, monotonicity and repeated policy evaluation imply

$$J_{\mu_{i+1}} \leq J_{\mu_i} + \frac{c}{1-\alpha} \mathbf{1}.$$

Substitute this inequality in $J_{\mu_{i+1}} = T_{\mu_{i+1}} J_{\mu_{i+1}}$ and compare with $T J^* = J^*$ to obtain

$$\|J_{\mu_{i+1}} - J^*\|_\infty \leq \alpha \|J_{\mu_i} - J^*\|_\infty + \frac{c}{1-\alpha}.$$

Iterating proves Equation (4.32). For Equation (4.33), eventual constancy means that a further approximate improvement returns $\bar{\mu}$ itself. The same comparison, together with its exact policy equation, gives

$$T J_{\bar{\mu}} \leq J_{\bar{\mu}} = T_{\bar{\mu}} J_{\bar{\mu}} \leq T J_{\bar{\mu}} + c \mathbf{1}.$$

Hence $\|J_{\bar{\mu}} - T J_{\bar{\mu}}\|_\infty \leq c$. By contraction,

$$\|J_{\bar{\mu}} - J^*\|_\infty \leq c + \|T J_{\bar{\mu}} - T J^*\|_\infty \leq c + \alpha \|J_{\bar{\mu}} - J^*\|_\infty.$$

Rearranging gives $\|J_{\bar{\mu}} - J^*\|_\infty \leq c/(1-\alpha)$. These are the bounds developed in Bertsekas (2019, Sec. 4.6.3). $\square$

The bound in Equation (4.32) describes an *error region*, not convergence of the policies. Even with a finite policy set, approximate PI may alternate among policies whose evaluation errors reverse a few close action comparisons. This is one reason a low critic RMSE does not imply

monotone policy improvement. The bound in Equation (4.33) explains why an eventually fixed policy is materially different from an oscillating sequence, although stability alone still does not prove optimality. The factors involving $(1 - \alpha)^{-1}$ also show why long-horizon problems demand either small uniform errors or additional structure; the bound is intentionally conservative and may be much larger than empirical loss.

These asymptotic bounds compare policies with the unknown optimum. For a *single improvement step*, the practical question is different: can the candidate be worse than the current policy because its evaluation is inaccurate? If $\bar{\mu}$ is exactly greedy with respect to $\tilde{J}_\mu$ and $\|\tilde{J}_\mu - J_\mu\|_\infty \leq \delta$, then

$$T_{\bar{\mu}}J_\mu \leq J_\mu + 2\alpha\delta\mathbf{1}, \quad (4.34)$$

$$J_{\bar{\mu}} \leq J_\mu + \frac{2\alpha\delta}{1-\alpha}\mathbf{1}. \quad (4.35)$$

To see the factor $2\alpha\delta$, compare two controls using one-step costs followed by J_μ . Replacing J_μ by $\tilde{J}_\mu$ changes each control's expected cost by at most $\alpha\delta$. Their difference can therefore change by as much as $2\alpha\delta$: a small apparent improvement can be only evaluation error. For example, with $\alpha = 0.9$ and $\delta = 0.1$, an estimated one-step cost reduction of 0.05 is smaller than the possible comparison error 0.18.

Equation (4.34) bounds this one-step deterioration when the candidate action is followed by μ . Repeating the candidate policy at every step accumulates the allowance geometrically, producing $2\alpha\delta/(1 - \alpha)$ in Equation (4.35). These are upper bounds on possible deterioration, not predictions of a loss. Unlike exact policy improvement, they do not guarantee $J_{\bar{\mu}} \leq J_\mu$, nor do they assign a probability to improvement. When $\delta = 0$, they recover non-worsening, not necessarily strict improvement. Independent policy evaluation is therefore needed to establish an actual improvement; a lower fitting loss alone is insufficient.

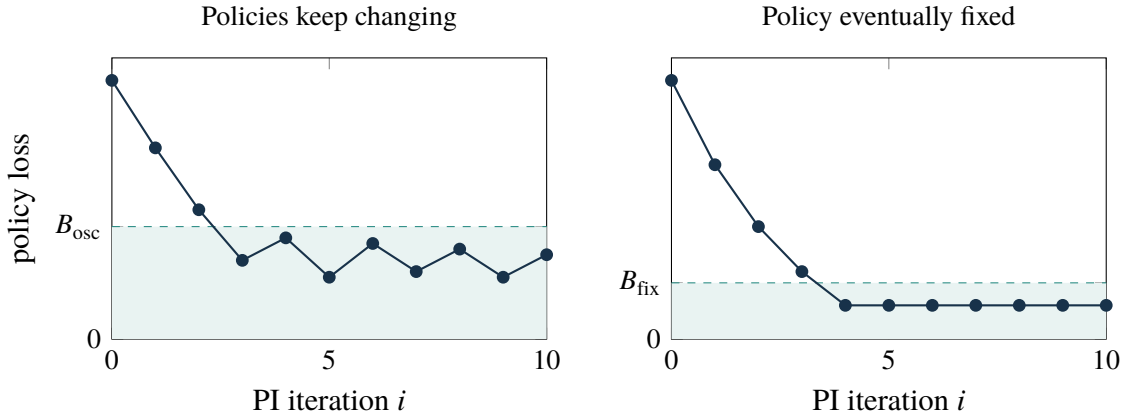


Figure 4.9: Schematic policy-loss sequences, not simulation results. $B_{\text{osc}} = (\epsilon + 2\alpha\delta)/(1 - \alpha)^2$ and $B_{\text{fix}} = (\epsilon + 2\alpha\delta)/(1 - \alpha)$ are the two bounds. The plotted heights are illustrative: neither initial monotonic progress nor oscillation is required by the theorem. Reconstructed from Bertsekas (2019, Figs. 4.6.4–4.6.5).

Figure 4.9 distinguishes bounded policy loss from convergence of the policy sequence. It also explains why stopping solely on a critic’s training loss can be misleading.

Architecture, cost shaping, and policy oscillation.

The implementation choices in Bertsekas (2019, Sec. 4.7.4) connect directly to Chapter 3. Fixed features give linear least-squares evaluations; neural features permit richer representations but add optimization error. A useful nominal function V can instead be separated from the learned correction. For bounded V , define

$$g_V(x, u, w) = g(x, u, w) + \alpha V(f(x, u, w)) - V(x).$$

Discounted summation telescopes, giving $J_\mu^V(x) = J_\mu(x) - V(x)$. Exact policy rankings are unchanged, but fitting the residual $J_\mu - V$ may be easier for a chosen architecture. This is *cost shaping*, not a change in the intended optimal-control objective.

Coverage must include states and alternative controls relevant to the next policy, not only the current policy’s common trajectories. If policy iterations oscillate, independently evaluate candidate policies and inspect the close action comparisons driving the changes. Damping updates or keeping the best validated candidate may help in practice, but does not establish the uniform-error assumptions of the proposition.

4.5.2 Why approximate PI is not merely fitted VI with different labels

Fitted VI repeatedly approximates the optimality map $\tilde{J}_{i+1} \approx T\tilde{J}_i$. Approximate PI instead freezes a policy long enough to approximate the affine policy map T_{μ_i} , and only then changes that policy. The distinction has practical consequences, summarized in Table 4.1.

Neither method uniformly dominates the other. Policy evaluation can be easier than global optimality approximation because it follows one policy’s trajectories, but improvement needs reliable comparisons for actions that the current policy may rarely take. Approximate VI avoids an explicit outer loop, but projection is applied to a moving optimality target and the projected map need not be a contraction. Approximate PI makes the control step visible, which is useful when a mobility controller must restrict maneuvers, preserve a baseline, or audit every proposed policy. The data requirement includes two different uses: following the current policy provides evaluation data, but comparing alternatives requires information about controls it may not choose. With a J evaluator, a model or simulator predicts their costs and successors; with a Q evaluator, the fitted Q -factor must reliably cover those controls. Thus improvement does not always require a separate set of full trajectories, but data containing only the current policy’s controls may be insufficient for a model-free Q comparison. Neither route is reliable merely because the dataset is large.

This book gives particular attention to the PI organization because a current or base policy provides a concrete reference for improvement. Its approximate cost-to-go can be used in online lookahead or rollout, connecting learned evaluation to model-based control in Section 4.7 and Chapter 5. That connection, rather than a claim that approximate PI always outperforms approximate VI, motivates the emphasis.

Table 4.1: Direct approximate VI and approximate PI compared.

Question	Fitted/approximate VI	Approximate PI
Target object	optimal value J^* or Q^*	current-policy value J_{μ_i} or Q_{μ_i}
Target operator	nonlinear minimum in every backup	fixed-policy operator during evaluation
Information needed	costs and successors for comparing candidate controls at fitted states	current-policy evaluation data; model predictions or Q estimates for comparing controls
Control update	implicit in every target	explicit improvement after evaluation
Typical risk	projection error is re-injected every VI sweep	evaluation and improvement errors can cause policy oscillation
What helps reliability	control of fitting errors across successive Bellman targets	evaluation errors small enough to distinguish candidate actions

4.5.3 Sample-Based Optimistic Approximate PI

Fully evaluating every intermediate policy can spend substantial computation on a policy that will soon be replaced. The evaluation block in Section 4.5 therefore need not be solved to convergence before improvement. *Optimistic PI* deliberately uses a limited amount of evaluation and then changes the policy. The term refers to incomplete evaluation, not optimistic initialization or exploration bonuses (Bertsekas 2019, Sec. 4.5.2). More than one evaluation update is permitted, but is not what makes the method optimistic: even one update may be used.

Here we implement that partial evaluation using sampled transitions and function fitting instead of exact Bellman expectations. This is *sample-based* optimistic approximate PI: the evaluation budget is limited, and each evaluation update is itself approximate. The samples may come from observed operation or a simulator. *Simulation-based*, as used in Bertsekas (2019, Secs. 4.5.2 and 4.7), specifies that these samples are generated by a simulator. The following construction can use either source, provided the data evaluate the stated policy.

Refining the evaluator while holding the policy fixed.

At outer iteration i , fix μ_i and initialize an evaluator $\tilde{J}_{i,0}$, for example from the preceding iteration. Let j count inner evaluation updates. Collect transitions $(x^{(s)}, \mu_i(x^{(s)}), g^{(s)}, x^{+(s)})$ under that policy and form

$$y_{i,j}^{(s)} = g^{(s)} + \alpha \tilde{J}_{i,j}(x^{+(s)}), \quad r_{i,j+1} \in \arg \min_r \sum_s [\tilde{J}(x^{(s)}; r) - y_{i,j}^{(s)}]^2,$$

with $\tilde{J}_{i,j+1}(x) = \tilde{J}(x; r_{i,j+1})$. This approximates a fixed-policy Bellman evaluation step, not an optimality backup: the sampled control follows μ_i and no minimization over controls occurs inside its target.

After $m_i \geq 1$ such updates, improve the policy using $\tilde{J}_{i,m_i}$, without requiring it to equal J_{μ_i} :

$$\mu_{i+1}(x) \in \arg \min_{u \in \mathcal{U}(x)} \mathbb{E}[g(x, u, w) + \alpha \tilde{J}_{i,m_i}(f(x, u, w)) \mid x, u], \quad x \in \mathcal{X}.$$

This displayed improvement uses a model; a Q-factor implementation can instead minimize the fitted Q-function directly as discussed above.

Algorithm lens: Sample-based optimistic approximate PI

1. Fix the current policy μ_i and initialize its evaluator.
2. Perform a chosen finite number m_i of sampled evaluation updates, refreshing the bootstrap targets from the updated evaluator.
3. Improve the policy from that partial evaluation and carry the evaluator forward as an initial guess for the next outer iteration.

The updated cost-to-go at the successor makes each inner evaluation step use the result of the preceding step. This is still partial evaluation of the same policy μ_i . Without sampling or fitting errors, m_i such steps give $T_{\mu_i}^{m_i} \tilde{J}_{i,0}$: the expected sum of the first m_i discounted costs under μ_i , followed by $\alpha^{m_i} \tilde{J}_{i,0}(x_{m_i})$ for the remaining cost. One can therefore also estimate this quantity from length- m_i trajectories and fit it once. This is an alternative implementation of partial evaluation, not an exact equivalence to repeated fitting, which introduces errors at each inner step.

By contrast, repeatedly optimizing one regression loss with unchanged trajectory-return labels merely fits the same labels more accurately. It does not constitute additional Bellman evaluation steps. In the earlier trajectory formulas, a fixed terminal approximation can define such a fixed-label fit; for iterative bootstrapping, the terminal evaluator must be updated according to the chosen evaluation scheme. Finite data, fitting error, and early stopping still matter, so the exact optimistic-PI convergence result in Chapter 2 does not automatically carry over to this sample-based approximation.

4.6 Sampled Policy Evaluation and On-Policy Control

The approximate-PI loop in Section 4.5 requires an evaluator for its current policy. We now develop MC and TD implementations of that block, then couple sampled Q-factor evaluation to improvement in SARSA. For a fixed policy μ , estimate

$$J_{\mu}(x) = \mathbb{E}_{\mu} \left[\sum_{t=0}^{\infty} \alpha^t g(x_t, u_t, w_t) \mid x_0 = x \right]. \quad (4.36)$$

This is the critic step of policy iteration. The data distribution is induced by the initial-state distribution, the policy, and the dynamics; it is not automatically the distribution required for safe policy improvement.

How much evaluation before improvement?

MC and TD do not prescribe a universal number of episodes or updates. In approximate PI, hold the current policy fixed while collecting evaluation data and fitting its cost-to-go, then use that evaluator in a separate improvement step. A practical evaluation budget combines a sample or computation limit with checks on additional data: do predicted returns and the resulting action comparisons change materially when more data or updates are added? These are stopping diagnostics, not certificates of accurate evaluation at every state. MC can spend the budget on more trajectory returns; TD can also refine its bootstrap estimates through successive updates. Waiting for complete evaluation is not required: Section 4.5.3 describes deliberately limited evaluation before each policy improvement. Too little evaluation can make the next policy respond mainly to estimation error, whereas evaluating a policy far beyond the accuracy needed for its next improvement wastes work.

4.6.1 Monte Carlo returns

A *complete return* sums all subsequent discounted costs: to a genuine zero-cost terminal state in an episodic problem, or over the infinite future in a continuing problem. The latter cannot be observed in finite time. A *truncated return* stops after N transitions and replaces the unobserved remainder by a terminal estimate:

$$G_k^{(N)} = \sum_{t=0}^{N-1} \alpha^t g_{k+t} + \alpha^N \hat{J}_T(x_{k+N}). \quad (4.37)$$

Here $g_t = g(x_t, u_t, w_t)$ is the sampled stage cost, and $\hat{J}_T$ is a declared terminal evaluator; the subscript prevents it from being confused with the policy value currently being learned. With $\hat{J}_T = 0$, a sufficiently long trajectory approximates the full discounted return. With $\hat{J}_T \approx J_\mu$, shorter rollouts can be used, at the price of terminal-target bias. A tabular every-visit update is

$$\tilde{J}_\mu(x_k) \leftarrow \tilde{J}_\mu(x_k) + \eta_k (G_k^{(N)} - \tilde{J}_\mu(x_k)). \quad (4.38)$$

Here $\eta_k > 0$ is a stepsize. For a parameterized value, the corresponding squared-error update is

$$\theta^+ = \theta + \eta_k (G_k^{(N)} - \tilde{J}_\mu(x_k; \theta)) \nabla_\theta \tilde{J}_\mu(x_k; \theta).$$

The return is held fixed during this update. A complete return is an MC target; a truncated return with a learned terminal evaluator also bootstraps.

The return uses actual sampled costs and transitions, so neither MC nor the TD methods below require an analytic transition model. In this precise backup sense they are *model free*. They can still consume trajectories produced by a model-based simulator, and a later greedy improvement from a state-value critic may again require a model.

An unweighted sampled return evaluates the policy that generated its actions. This is *on-policy evaluation* when that behavior policy is also the policy we wish to evaluate. Reusing trajectories from a different policy instead requires *off-policy evaluation*. Importance sampling provides one way to correct this mismatch without collecting new trajectories. If trajectories were generated

by a behavior policy μ_{beh} but the target is another stochastic policy μ , a trajectory likelihood ratio is (using probability densities for continuous actions when the required density ratio exists)

$$\varrho_{k:k+N-1} = \prod_{t=k}^{k+N-1} \frac{\mu(u_t | x_t)}{\mu_{\text{beh}}(u_t | x_t)}. \quad (4.39)$$

For a fixed starting state, a trajectory's probability is the product of its action probabilities and transition probabilities. Under the same system, the transition factors cancel in the ratio of target to behavior trajectory probabilities, leaving Equation (4.39). Weighting by this ratio therefore converts a behavior-policy expectation to a target-policy expectation. For a fixed terminal evaluator,

$$\mathbb{E}_{\mu_{\text{beh}}}[\varrho_{k:k+N-1} G_k^{(N)} | x_k = x] = \mathbb{E}_{\mu}[G_k^{(N)} | x_k = x].$$

This corrects the policy mismatch after x_k , not the distribution of starting states or error in the terminal estimate. It requires support— $\mu_{\text{beh}}(u | x) > 0$ whenever $\mu(u | x) > 0$ —and products of many ratios may have enormous variance. Thus data reuse comes at a possible variance cost. Per-decision importance sampling weights the cost g_{k+t} only by action ratios from k through $k+t$: actions taken later cannot affect a cost already incurred. This avoids multiplying that cost by unnecessary later ratios while preserving its target-policy expectation. Clipping large ratios can stabilize the calculation, but generally introduces bias.

Trajectory reuse and the bias–variance tradeoff.

A single trajectory can provide labels for several visited states. Starting with $G_N = \hat{J}_T(x_N)$, compute backward

$$G_t = g_t + \alpha G_{t+1}, \quad t = N-1, \dots, 0.$$

Then (x_t, G_t) is a training pair whose remaining simulation length is $N-t$. These labels share costs and are correlated; they are not independent new trajectories. If $\|\hat{J}_T - J_{\mu}\|_{\infty} \leq d$, the conditional bias from truncation is bounded by $\alpha^{N-t}d$. Labels near the end depend more strongly on the terminal approximation.

For a fixed simulation budget, shorter trajectories permit more starting states and often reduce return variance, but increase dependence on the terminal estimate. Longer trajectories reduce terminal bias but concentrate samples on states visited by the current policy. The actual variance depends on the dynamics and temporal correlations. This tradeoff motivates both multistep TD and deliberate exploration.

4.6.2 One-step temporal difference

TD(0) replaces the unknown cost-to-go by the current approximator. Write $g_k = g(x_k, u_k, w_k)$ for the observed stage cost and let $\eta_k > 0$ be the stepsize:

$$\underset{\text{one-step TD target}}{y_k^{\text{TD}}} = g_k + \alpha \tilde{J}_{\mu}(x_{k+1}; \theta_k), \quad (4.40)$$

$$\delta_k = y_k^{\text{TD}} - \tilde{J}_{\mu}(x_k; \theta_k), \quad (4.41)$$

$$\theta_{k+1} = \theta_k + \eta_k \delta_k \nabla_{\theta} \tilde{J}_{\mu}(x_k; \theta_k). \quad (4.42)$$

Here k is both a physical sample time and an update index because one update is made from each transition. The update is often called a semi-gradient: the bootstrap target depends on θ_k , but its derivative is not followed in Equation (4.42). TD can update before a trajectory terminates and often has lower target variance than a long return, but it introduces bootstrap bias and can propagate errors from successor states.

Like MC, this update uses the observed tuple (x_k, u_k, g_k, x_{k+1}) rather than $p(x' | x, u)$, so on-policy TD evaluation is model free in its target construction. Unlike MC, it bootstraps. Unlike Q-learning, it does not contain an optimality minimum and therefore does not improve a policy by itself.

Worked Example 4.2 (A common trajectory viewed by MC and TD). Consider a fixed policy on two states with deterministic transitions $x^0 \rightarrow x^1$ at cost 1 and $x^1 \rightarrow x^1$ at cost 2, with $\alpha = 0.9$. Its exact values are

$$J_\mu(x^1) = \frac{2}{1-0.9} = 20, \quad J_\mu(x^0) = 1 + 0.9(20) = 19.$$

Suppose the current critic has $\tilde{J}_\mu(x^1) = 10$. The one-step TD target from x^0 is $1 + 0.9(10) = 10$. A three-step truncated return with zero terminal value is

$$1 + 0.9(2) + 0.9^2(2) = 4.42,$$

which is strongly biased downward by artificial truncation. Using the exact terminal value after those three costs gives

$$1 + 0.9(2) + 0.9^2(2) + 0.9^3(20) = 19.$$

The same observed trajectory therefore supports different targets depending on the bootstrap and terminal convention. TD needs no transition matrix, but its target inherits the current critic error. Truncated MC also needs no transition matrix, but its target inherits terminal-value error. Neither target changes the policy until an improvement step is added.

4.6.3 The continuum between MC and TD(0)

One-step TD uses little observed future information, whereas MC waits for a complete return or uses a long truncation. Multistep TD lets the evaluator choose how much observed cost to use before bootstrapping. An n -step target is

$$G_k^{(n)} = \sum_{t=0}^{n-1} \alpha^t g_{k+t} + \alpha^n \tilde{J}_\mu(x_{k+n}; \theta). \quad (4.43)$$

Small n emphasizes bootstrapping; large n approaches a Monte Carlo return. The forward-view λ -return combines all depths,

$$G_k^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_k^{(n)}, \quad 0 \leq \lambda < 1, \quad (4.44)$$

The weights sum to one. At $\lambda = 0$ only the one-step target remains; larger λ puts more weight on longer returns. This definition applies to the continuing discounted problem as well as to episodic problems. With bounded costs and a bounded, fixed bootstrap function, the term $\alpha^n \tilde{J}_\mu(x_{k+n}; \theta)$ vanishes as $n \rightarrow \infty$. Consequently $G_k^{(n)}$ approaches the complete discounted return, and so does G_k^λ as $\lambda \uparrow 1$. This is a limit, not substitution of $\lambda = 1$ into Equation (4.44).

If a genuine terminal state is reached after H transitions, assign the remaining weight to the complete return:

$$G_k^\lambda = (1 - \lambda) \sum_{n=1}^{H-1} \lambda^{n-1} G_k^{(n)} + \lambda^{H-1} G_k^{(H)}.$$

Here the terminal cost-to-go is zero, so $\lambda = 1$ gives the complete MC return exactly; an artificial cutoff instead retains any error in its terminal estimate.

The return formula looks forward from state x_k . To see how it leads to an update as each new transition arrives, temporarily hold θ fixed and write

$$\delta_t(\theta) = g_t + \alpha \tilde{J}_\mu(x_{t+1}; \theta) - \tilde{J}_\mu(x_t; \theta).$$

For each depth n , successive bootstrap terms cancel:

$$G_k^{(n)} - \tilde{J}_\mu(x_k; \theta) = \sum_{j=0}^{n-1} \alpha^j \delta_{k+j}(\theta).$$

Combining these identities with the λ -return weights gives, for the continuing discounted case,

$$G_k^\lambda - \tilde{J}_\mu(x_k; \theta) = \sum_{t=k}^{\infty} (\alpha \lambda)^{t-k} \delta_t(\theta).$$

Thus an error observed at time t contributes to earlier states with a weight that decreases with their age. An *eligibility trace* stores these decaying contributions, so they need not be recomputed from the whole history. For a differentiable critic, the accumulating trace and semi-gradient update are

$$z_k = \alpha \lambda z_{k-1} + \nabla_{\theta} \tilde{J}_\mu(x_k; \theta_k), \quad z_{-1} = 0, \quad (4.45)$$

$$\theta_{k+1} = \theta_k + \eta_k \delta_k z_k. \quad (4.46)$$

Expanding the recursion gives $z_k = \sum_{t=0}^k (\alpha \lambda)^{k-t} \nabla_{\theta} \tilde{J}_\mu(x_t; \theta_t)$: the current TD error updates parameters associated with both the current and recently visited states. This explains the connection between the forward return and the trace-based update. The displayed return identity holds for fixed parameters; ordinary online TD(λ), which changes parameters after each transition, is not exactly the same finite-data computation. At $\lambda = 0$, z_k contains only the current gradient, so Equation (4.46) is exactly TD(0). As λ approaches one in a terminating task, the forward target approaches the Monte Carlo return, subject to the chosen terminal convention. Thus TD(0) is not a separate family from TD(λ); it is its one-step endpoint.

For comparison with projected policy evaluation below, take the conditional expectation of the sampled return. If the fixed bootstrap function is J , then $\mathbb{E}_\mu[G_k^{(n)} \mid x_k = x] = (T_\mu^n J)(x)$. Consequently the expected λ -return is

$$T_\mu^{(\lambda)} J = (1 - \lambda) \sum_{n=0}^{\infty} \lambda^n T_\mu^{n+1} J. \quad (4.47)$$

In particular, $(T_\mu^{(\lambda)} J)(x) = \mathbb{E}_\mu[G_k^\lambda \mid x_k = x]$ when that return uses J for bootstrapping. This is the expectation form of Equation (4.44), not a different return definition. Larger λ reduces bootstrapping but commonly raises sampling variance and trace length. The MC, TD, and trace formulations follow the standard development in Sutton and Barto (2018, Chaps. 6–7 and 12); the signs here use cost minimization rather than reward maximization. The projected-equation view and the comparison of TD, LSTD, and LSPE follow Bertsekas (2019, Sec. 4.9).

Terminology note

“Bias versus variance” is useful but incomplete. A one-step target can have low conditional variance and still be systematically wrong because its bootstrap value is wrong. A long Monte Carlo return can be unbiased for the behavior policy yet irrelevant to a different target policy. Coverage, partial observability, truncation, and policy mismatch must be reported along with target variance.

4.6.4 Direct and Indirect Linear Policy Evaluation

For $\tilde{J}_\mu(x; r) = \phi(x)^\top r$, define the projection of a function h onto the feature class by

$$(\Pi_\nu h)(x) = \phi(x)^\top r_h, \quad r_h \in \arg \min_r \mathbb{E}_{x \sim \nu} [(\phi(x)^\top r - h(x))^2].$$

Here ν weights the states where approximation accuracy matters. In a finite-state problem the expectation is a weighted sum, and the feature matrix Φ has rows $\phi(x)^\top$. A *direct* method targets $\Phi r = \Pi_\nu J_\mu$: the best fit to the policy’s actual cost-to-go under this weighting. An *indirect* method instead seeks consistency with its own projected Bellman backup,

$$\Phi r = \Pi_\nu T_\mu(\Phi r). \quad (4.48)$$

The unknown approximation appears on both sides of this fixed-point equation. Its solution generally differs from $\Pi_\nu J_\mu$. The matrix notation is convenient for displaying the projection; its expectation definition above also applies to continuous state spaces.

Direct evaluation by MC regression.

Use the sampled returns as regression labels:

$$r_{\text{MC}} \in \arg \min_r \sum_{k=0}^{q-1} \left(\phi(x_k)^\top r - G_k^{(N)} \right)^2. \quad (4.49)$$

With complete on-policy returns and adequate sampling, this approximates $\Pi_\nu J_\mu$; truncation can introduce terminal bias. The remaining methods target Equation (4.48) instead. We use the suffix (0) because their bootstrap uses one transition, without a multistep trace.

TD(0): one transition per update.

Substituting the linear architecture into Equation (4.42) gives

$$r_{k+1} = r_k + \eta_k \phi(x_k) [g_k + \alpha \phi(x_{k+1})^\top r_k - \phi(x_k)^\top r_k]. \quad (4.50)$$

This equation is the simple critic update that is easily obscured by a deep-RL software stack: compute one scalar TD error, multiply by the active feature vector, and add the result to the parameters. It is *incremental* because each transition produces an update; the method's name remains TD(0). With on-policy stationary sampling, finite required moments, and linearly independent features on the support of ν , the mean update vanishes at the projected equation:

$$\mathbb{E}_\nu [\phi(x) ((T_\mu \tilde{J}_\mu)(x) - \tilde{J}_\mu(x; r))] = 0.$$

This is a target equation; convergence of the sampled iteration also requires suitable sampling and stepsize conditions.

LSTD(0): solve the sampled projected equation.

Least-squares temporal difference (LSTD) evaluation accumulates transitions to solve the projected fixed-point equation

$$\Phi r = \Pi_\nu T_\mu(\Phi r).$$

The same unknown r appears on both sides: LSTD seeks a self-consistent parameter vector by solving a linear system, rather than repeatedly applying the projected backup as LSPE does below. This is the population, unregularized target; the sampled system and optional regularization follow. To derive that system, first fix a bootstrap parameter ν and fit a separate parameter z :

$$z(\nu) \in \arg \min_z \mathbb{E}_\nu [(\phi(x)^\top z - g - \alpha \phi(x^+)^\top \nu)^2].$$

The expectation includes a transition under μ ; g and x^+ are its cost and successor. This regression defines the common projected-backup operation $\Phi z(\nu) = \Pi_\nu T_\mu(\Phi \nu)$; LSTD seeks its fixed point $r = z(r)$. Holding ν fixed, the least-squares normal equation is

$$\mathbb{E}_\nu [\phi(x) (\phi(x)^\top z - g - \alpha \phi(x^+)^\top \nu)] = 0.$$

At a projected fixed point, set $z = \nu = r$. Collecting its coefficients gives the population equation

$$Ar = b, \quad A = \mathbb{E}_\nu [\phi(x) (\phi(x) - \alpha \phi(x^+))^\top], \quad b = \mathbb{E}_\nu [\phi(x) g].$$

The two-step argument matters: differentiating both occurrences of the same parameter in a squared TD error would define a different method. Now replace these expectations by sums over q transitions, using $\phi_k = \phi(x_k)$:

$$A_q = \sum_{k=0}^{q-1} \phi_k (\phi_k - \alpha \phi_{k+1})^\top, \quad b_q = \sum_{k=0}^{q-1} \phi_k g_k, \quad (4.51)$$

$$r_{\text{LSTD}} = (A_q + \lambda_{\text{reg}} I)^{-1} b_q. \quad (4.52)$$

The inverse is assumed to exist. With $\lambda_{\text{reg}} = 0$, this solves $A_q r = b_q$. An optional *diagonal shift* $\lambda_{\text{reg}} I$ increases every diagonal entry by $\lambda_{\text{reg}} \geq 0$ to address poor conditioning, but does not guarantee nonsingularity for every TD matrix. It changes the empirical TD equation; it is not ordinary ridge regression on fixed labels. The shift parameter is distinct from the trace parameter λ . In code, solve the linear system rather than forming its inverse.

LSPE(0): iterate the projected backup.

Least-squares policy evaluation (LSPE) uses the same regression operation and targets the same fixed point, but approaches it by iteration rather than a single system solve. Its unrelaxed population step is

$$\Phi r_{i+1} = \Pi_v T_\mu(\Phi r_i).$$

In the regression above, take $v = r_i$ and fit $r_{i+1} = z(r_i)$, then repeat. Thus LSTD solves $r = z(r)$, whereas LSPE iterates the map z . For a fixed batch of transitions, let

$$C_q = \sum_{k=0}^{q-1} \phi_k \phi_k^\top. \quad (4.53)$$

$$\hat{r}_{i+1} \in \arg \min_z \sum_{k=0}^{q-1} [\phi_k^\top z - g_k - \alpha \phi_{k+1}^\top r_i]^2.$$

The normal equation is $C_q \hat{r}_{i+1} = b_q + (C_q - A_q) r_i$. When C_q is invertible, rearrangement gives $\hat{r}_{i+1} = r_i + C_q^{-1} (b_q - A_q r_i)$. A relaxed update takes a fraction η_i of this step; with an optional diagonal shift it is

$$r_{i+1} = r_i + \eta_i (C_q + \lambda_{\text{reg}} I)^{-1} (b_q - A_q r_i). \quad (4.54)$$

The unshifted step with $\eta_i = 1$ is precisely the sampled regression above. Here a diagonal shift changes the scaling matrix, not the residual $b_q - A_q r_i$. Thus, if the iteration converges, its fixed point still solves $A_q r = b_q$ and need not equal the shifted LSTD solution. Table 4.2 compares these routes. LSPE(0) can warm-start evaluation of a nearby policy, whereas LSTD(0) directly solves the accumulated system. Compared with TD(0), LSPE(0) uses more data per update and a feature covariance to scale the residual, at the cost of matrix storage and linear solves.

For multistep evaluation, the corresponding equation is $\Phi r = \Pi_v T_\mu^{(\lambda)}(\Phi r)$, using Equation (4.47). Linear TD(λ), LSTD(λ), and LSPE(λ) provide different numerical routes toward this equation. For $\lambda > 0$, traces also give earlier feature vectors a decaying weight when accumulating TD errors, as in Equation (4.45). Thus the linear architecture remains the same while the evaluation target uses longer returns. Detailed convergence claims require the sampling chain, feature rank, weighting distribution, and on/off-policy setting to be stated; the equations alone do not supply them.

4.6.5 SARSA: Interleaved Q-Factor Evaluation and Policy Improvement

MC and TD above evaluate a specified policy. To obtain a controller, evaluation must be coupled to policy improvement, as in Section 4.5. SARSA makes this coupling incremental: after a

Table 4.2: Linear policy evaluation with the same feature class. The LSTD(0) and LSPE(0) targets shown are unregularized; $\delta_k = g_k + \alpha \phi_{k+1}^\top r_k - \phi_k^\top r_k$.

Method	Target or update	Computation and tradeoff
MC regression	Fit sampled returns to approximate $\Pi_\nu J_\mu$	Least squares; complete returns do not bootstrap, but truncation can bias labels
TD(0)	$r_{k+1} = r_k + \eta_k \phi_k \delta_k$	One transition per update; low memory, but stepsize and sampling matter
LSTD(0)	$\Phi r = \Pi_\nu T_\mu(\Phi r)$, empirically $A_q r = b_q$	One system solve; no stepsize schedule, but matrix conditioning matters
LSPE(0)	$\Phi r_{i+1} = \Pi_\nu T_\mu(\Phi r_i)$	Repeated projected backups; permits warm starts, but needs matrix computations and convergence checks

sampled Q-factor update, the behavior policy is adjusted using the updated estimates rather than waiting for a complete evaluation.

SARSA abbreviates State–Action–Reward–State–Action; here costs replace rewards. With a fixed behavior policy, SARSA evaluates its Q-factor. When that policy is repeatedly made greedy or ϵ -greedy from the updated estimates, evaluation and improvement are interleaved. This is the extreme optimistic-PI interpretation in Bertsekas (2019, Sec. 4.8): very little evaluation occurs before the next policy change.

At time k , u_k has already been selected (at initialization for $k = 0$, or at the preceding step). After applying it and observing x_{k+1} , SARSA selects $u_{k+1} \sim \mu_{\text{beh}}(\cdot | x_{k+1})$ using the current table Q_k , before updating it. A common behavior policy is ϵ -greedy: with probability $1 - \epsilon$ choose a minimizing action, and with probability ϵ sample uniformly from the feasible actions. Pure greedy selection is the $\epsilon = 0$ case, but by itself need not explore sufficiently. The selected next action is also the action subsequently executed. Its Q-factor estimates the cost after the observed transition, so adding its discounted value to the observed cost g_k gives the target below. The update then moves the visited entry toward this target, leaving all other entries unchanged:

$$y_k^{\text{SARSA}} = g_k + \alpha Q_k(x_{k+1}, u_{k+1}), \quad (4.55)$$

$$Q_{k+1}(x_k, u_k) = Q_k(x_k, u_k) + \eta_k [y_k^{\text{SARSA}} - Q_k(x_k, u_k)]. \quad (4.56)$$

Expected SARSA replaces the sampled continuation action by an expectation under the same current behavior policy,

$$y_k^{\text{ExpSARSA}} = g_k + \alpha \sum_{v \in \mathcal{B}(x_{k+1})} \mu_{\text{beh}}(v | x_{k+1}) Q_k(x_{k+1}, v). \quad (4.57)$$

This averages over the next action in the *target*, not over the physical actions executed. The controller still selects and executes one action from its behavior policy; an ϵ -greedy policy still explores. The expectation removes that action-sampling variability from the target, but not the need to collect sufficiently diverse state–control data. Thus SARSA is on-policy when the same evolving policy generates the data and defines the continuation action. Q-learning is off-policy relative to its exploratory behavior data because it targets a greedy continuation regardless of which next action the behavior policy takes. “Off-policy” does not mean “without data from a policy”; it means that the target policy and behavior policy differ. The distinction is summarized in Table 4.3.

Algorithm 4.3: Tabular SARSA

Input: Finite state and feasible-control sets, discount $0 < \alpha < 1$, access to sampled system transitions and costs, an ϵ -greedy behavior rule with a prescribed exploration schedule, a pair-specific stepsize schedule, and update budget K .

Initialize: Choose finite values $Q_0(x, u)$ for every feasible state–control pair and an initial nonterminal state x_0 . Select u_0 from the behavior policy constructed from Q_0 .

For $k = 0, \dots, K - 1$:

1. **Collect a transition.** Apply the already selected u_k , and observe cost g_k and successor state x_{k+1} .
2. **Select the next action and compute the target.** If x_{k+1} is nonterminal, select u_{k+1} from the behavior policy constructed from Q_k at that state, and set

$$y_k^{\text{SARSA}} = g_k + \alpha Q_k(x_{k+1}, u_{k+1}).$$

At a genuine zero-cost terminal state, select no next action and set $y_k^{\text{SARSA}} = g_k$ instead.

3. **Update the visited entry.** Using the stepsize η_k assigned to this pair’s current visit, set

$$Q_{k+1}(x_k, u_k) = Q_k(x_k, u_k) + \eta_k [y_k^{\text{SARSA}} - Q_k(x_k, u_k)].$$

Set $Q_{k+1}(x, u) = Q_k(x, u)$ for every other feasible pair.

4. **Continue.** Carry the successor state and the *already selected* u_{k+1} into the next step; do not reselect this action after the update. Future action selections use the updated table. If the episode terminated, reset to a new initial state and select its first action using Q_{k+1} instead.

Output: The table Q_K and its greedy policy

$$\tilde{\mu}_K(x) \in \arg \min_{u \in \mathcal{U}(x)} Q_K(x, u) \quad \text{for every nonterminal state } x.$$

The following example isolates this distinction: the observed transition and current Q entry are identical, but the next action used for evaluation differs.

Worked Example 4.3 (Same transition, different continuation action). Let $g_k = 1$, $\alpha = 0.9$, $Q_k(x_k, u_k) = 5$, and let the next state's two action values be 2 and 6. The first action is greedy because we minimize cost. Under the ϵ -greedy rule with $\epsilon = 0.2$, the action probabilities are $(0.9, 0.1)$: the uniform exploration branch can select either action, including the greedy one. Q-learning always uses $y_k^Q = 1 + 0.9(2) = 2.8$ for this transition. SARSA uses the same target with probability 0.9, when the selected next action is greedy. With probability 0.1, the second action is selected and its target is $1 + 0.9(6) = 6.4$. For stepsize 0.1, the corresponding updated entries are 4.78 and 5.14. Thus SARSA usually agrees with the greedy-continuation target here, but it also accounts for the costly exploration that its behavior policy sometimes executes.

Expected SARSA uses the probability-weighted target $1 + 0.9[0.9(2) + 0.1(6)] = 3.16$, giving the updated entry 4.816. This is the mean of the two SARSA updates conditional on the observed transition and current table. It still describes the same exploratory policy, rather than removing exploration from the controller.

With a differentiable Q approximator, form the same SARSA target using $\tilde{Q}$ in place of the table. The corresponding semi-gradient update is

$$r_{k+1} = r_k + \eta_k [y_k^{\text{SARSA}} - \tilde{Q}(x_k, u_k; r_k)] \nabla_r \tilde{Q}(x_k, u_k; r_k).$$

For $\tilde{Q}(x, u; r) = \phi(x, u)^\top r$, the gradient is $\phi(x, u)$, and substitution gives the explicit feature-based update

$$r_{k+1} = r_k + \eta_k [g_k + \alpha \phi(x_{k+1}, u_{k+1})^\top r_k - \phi(x_k, u_k)^\top r_k] \phi(x_k, u_k). \quad (4.58)$$

The next-action feature term is omitted at a genuine zero-cost terminal state. As with Q-learning, function approximation does not inherit tabular convergence automatically. For a finite-state,

Table 4.3: Sampled Q updates under the cost-minimization convention.

Method	Continuation target	Policy relation	Main interpretation
Q-learning	$\min_v Q(x', v)$	Off-policy relative to exploratory data	Sampled optimality backup
SARSA	$Q(x', u'), u' \sim \mu_{\text{beh}}$	On-policy	Sampled policy-evaluation backup while policy changes
Expected SARSA	$\sum_v \mu_{\text{beh}}(v x') Q(x', v)$	On-policy expectation	Lower action-sampling variance with known behavior probabilities

finite-action discounted problem with bounded costs, tabular SARSA converges to Q^* under the standard conditions: every state–action pair is visited infinitely often, the behavior becomes greedy in the limit, and the stepsizes for each pair satisfy $\sum_j \eta_j(x, u) = \infty$ and $\sum_j \eta_j(x, u)^2 < \infty$. Here j counts visits to that pair. The first two conditions are called *greedy in the limit with infinite exploration* (GLIE). An ϵ_k -greedy policy can satisfy them if $\epsilon_k \rightarrow 0$ while maintaining infinite visitation; decreasing ϵ_k alone is not sufficient. See Singh et al. (2000).

With fixed $\epsilon > 0$, nongreedy actions remain possible even after much training. SARSA’s target therefore still includes the cost of future exploration, rather than only the greedy continuation used by Q-learning. The GLIE guarantee of convergence to the unrestricted Q^* consequently does not apply to fixed- ϵ SARSA.

4.7 Online Policy Improvement: Multistep Lookahead and Rollout

The PI organization in Section 4.5 first evaluates a policy and then improves its decisions. Online lookahead uses the same separation at the current state: a learned cost-to-go or a base policy evaluates the remaining trajectory, while online search selects the next control. Improved actions may be executed directly or collected as training labels for a new policy. Chapter 5 develops the geometric interpretation; here we focus on the computation and its guarantees.

In the lookahead and rollout setting of Bertsekas (2019, Sec. 4.6), *online planning* means computing a control from predicted future costs at the current state during execution. The learned functions remain fixed within that calculation. *Online policy improvement* describes using this computation to improve the decisions available from an approximate value or a base policy. Thus planning names the computation, whereas improvement names its intended effect on closed-loop cost. Improvement is guaranteed only under suitable evaluation and search conditions; it is not implied by performing a search or by updating learned parameters.

A value approximation can close a finite planning horizon at its *leaf states*, the endpoint states at which explicit prediction stops. Given a state x_k , an ℓ -step stochastic lookahead computes

$$\min_{\mu_0, \dots, \mu_{\ell-1}} \mathbb{E} \left[\sum_{t=0}^{\ell-1} \alpha^t g(x_{k+t}, \mu_t(x_{k+t}), w_{k+t}) + \alpha^\ell \tilde{J}(x_{k+\ell}) \mid x_k \right] \quad (4.59)$$

The minimization is over admissible state-feedback rules, and the controller applies only the first control. Admissibility must encode the intended constraints: an accurate critic alone does not establish constraint satisfaction in the physical system. In operator form its root value is $T^\ell \tilde{J}$. The terminal approximation compresses the distant future; the online Bellman steps recompute near-term actions using the current state and model.

The first question is how the error in $\tilde{J}$ affects the cost of the resulting policy when lookahead is repeated at every state. The next proposition answers this question relative to J^* , assuming a known uniform approximation error. It complements proposition 4.2: that result concerns a sequence of approximate PI steps whose evaluators approximate the current policy cost J_{μ_i} . Here there is one terminal evaluator approximating J^* and one induced lookahead policy; the proposition does not assume that approximate PI produced that evaluator.

Proposition 4.3 (Greedy policy induced by an approximate value). *Under assumption 4.1, suppose $\|\tilde{J} - J^*\|_\infty \leq \varepsilon$. Let $\tilde{\mu}$ be greedy with respect to $\tilde{J}$. Then*

$$\|J_{\tilde{\mu}} - J^*\|_\infty \leq \frac{2\alpha}{1-\alpha} \varepsilon. \quad (4.60)$$

If $\tilde{\mu}_\ell$ is the first-stage policy from exact ℓ -step lookahead with terminal $\tilde{J}$, then

$$\|J_{\tilde{\mu}_\ell} - J^*\|_\infty \leq \frac{2\alpha^\ell}{1-\alpha} \varepsilon. \quad (4.61)$$

Proof. The greedy relation is $T_{\tilde{\mu}}\tilde{J} = T\tilde{J}$. Changing J^* to $\tilde{J}$ in either operator changes its value by at most $\alpha\varepsilon$. Insert the greedy equality between these two comparisons and use $TJ^* = J^*$ to obtain

$$\|T_{\tilde{\mu}}J^* - J^*\|_\infty \leq 2\alpha\varepsilon.$$

The residual bound for the contraction $T_{\tilde{\mu}}$ gives Equation (4.60). For ℓ -step lookahead, the first action is greedy with respect to $T^{\ell-1}\tilde{J}$, whose distance from J^* is no greater than $\alpha^{\ell-1}\varepsilon$. Substitution into the one-step result proves the second bound. $\square$

The powers of α explain why exact lookahead can attenuate terminal- value error. They do not account for an approximate model, restricted action search, Monte Carlo tree error, or real-time truncation. Each of those adds a separate error source. Nor does the bound say that deeper search is always worth its computation.

Proposition 4.3 bounds suboptimality relative to J^* ; it does not compare with a particular previously stored policy. Nor does a smaller bound for larger ℓ prove that every increase in depth improves the actual policy cost. The base-policy comparison is established separately in Section 4.7.2.

Constant offsets in the terminal approximation.

Adding a constant b to $\tilde{J}$ does not change the lookahead policy: every candidate receives the same additional discounted terminal cost $\alpha^\ell b$. This matters because a uniformly biased evaluator can still rank every action correctly; for example, $\tilde{J} = J^* + b$ induces an optimal policy despite its nonzero value error. Accordingly, the bounds in proposition 4.3 can use $\inf_{b \in \mathbb{R}} \|\tilde{J} + b - J^*\|_\infty$ instead of $\|\tilde{J} - J^*\|_\infty$. This is the control-relevant offset invariance discussed in Chapter 3.

Restricted control searches.

A different practical issue is that online computation may compare only a subset of feasible controls. The next result concerns such a restricted search, not the constant offset b . Its constant c bounds how much the best candidate's one-step score can exceed the current evaluator. This yields an upper bound on the resulting policy cost without knowing J^* .

Proposition 4.4 (Restricted one-step lookahead). *Under assumption 4.1, let $\overline{\mathcal{U}}(x) \subseteq \mathcal{U}(x)$ be nonempty, with an attained measurable minimizing policy $\bar{\mu}$. Define*

$$B(x) = \min_{u \in \overline{\mathcal{U}}(x)} \mathbb{E}[g(x, u, w) + \alpha \tilde{J}(f(x, u, w)) \mid x, u].$$

If $B(x) \leq \tilde{J}(x) + c$ for every x , where $c \in \mathbb{R}$, then

$$J_{\bar{\mu}}(x) \leq \tilde{J}(x) + \frac{c}{1 - \alpha}, \quad x \in \mathcal{X}.$$

Proof. $T_{\bar{\mu}} \tilde{J} \leq \tilde{J} + c$. Monotonicity and the discounted constant-shift identity give $T_{\bar{\mu}}^n \tilde{J} \leq \tilde{J} + c \sum_{t=0}^{n-1} \alpha^t$. Taking $n \rightarrow \infty$ proves the result. $\square$

The proof uses the restricted-search residual argument of Bertsekas (2019, Prop. 4.6.1(b) and its appendix proof). Unlike proposition 4.3, it needs neither J^* nor an error estimate relative to J^* . The quantity $B(x) - \tilde{J}(x)$ measures how much one restricted Bellman step raises the evaluator; a uniform upper bound c controls how this discrepancy can accumulate over repeated decisions. For example, $c = 0.02$ with $\alpha = 0.9$ gives $J_{\bar{\mu}}(x) \leq \tilde{J}(x) + 0.2$. This is useful as a policy-cost certificate even if $\tilde{J}$ is not the cost of any base policy. It is a worst-case upper bound, not by itself a comparison with a previously used controller.

A stronger interpretation is available when $\tilde{J} = J_{\mu_{\text{base}}}$ and $\mu_{\text{base}}(x) \in \overline{\mathcal{U}}(x)$ for every x . The base control then supplies a candidate whose score is exactly $J_{\mu_{\text{base}}}(x)$, so $B \leq \tilde{J}$ and $c = 0$ is valid. The result now guarantees no worse cost than the base policy, as developed in Section 4.7.2.

The next example returns to the different bound in proposition 4.3, which compares a lookahead policy with the optimum. It explains why that global bound can be large without being mathematically loose in the worst case.

Worked Example 4.4 (A global bound can be attained). Consider the deterministic two-state problem of Bertsekas (2019, Example 4.6.1). State 2 is absorbing with zero stage cost. At state 1, the controller may *leave* for state 2 at zero cost or *stay* at state 1 at cost $2\alpha\epsilon$, where $\epsilon > 0$. Leaving immediately is optimal, so $J^* = (0, 0)$.

Now let $\tilde{J}(1) = -\epsilon$ and $\tilde{J}(2) = \epsilon$. The approximation underestimates the cost of staying and overestimates the cost after leaving. One-step lookahead therefore assigns the scores

$$\begin{aligned} \text{leave: } & 0 + \alpha \tilde{J}(2) = \alpha \epsilon, \\ \text{stay: } & 2\alpha \epsilon + \alpha \tilde{J}(1) = \alpha \epsilon. \end{aligned}$$

The two actions tie even though only leaving is optimal. If ties are resolved by staying, the same decision recurs at every step. Its actual infinite-horizon cost is consequently

$$J_{\bar{\mu}}(1) = \sum_{t=0}^{\infty} \alpha^t (2\alpha \epsilon) = \frac{2\alpha \epsilon}{1 - \alpha}.$$

Since $\|\tilde{J} - J^*\|_{\infty} = \epsilon$, this attains Equation (4.60). With $\alpha = 0.9$ and $\epsilon = 1$, the two scores are both 0.9, but staying forever costs 18 rather than 0. Choosing the other tied action produces

the optimal policy. Thus the bound allows a genuinely bad outcome; it does not predict that every approximation of this accuracy, or every tie-breaking rule, will produce it.

Global bounds versus local policy performance.

Bertsekas (2025, Sec. 1.5.3) emphasizes that the preceding linear bounds are worst-case envelopes, not predictions of actual performance. This does not invalidate them. In worked example 4.4, changing ε also changes the underlying control problem. It does not show that a fixed problem must incur policy loss proportional to arbitrarily small value error. Figure 4.10 makes the distinction explicit: approximation error is the horizontal axis, whereas the vertical axis is the cost loss of the policy obtained by using that approximation in lookahead. The straight line is an upper bound on this loss, not the loss itself.

To understand the difference, now hold the control problem fixed and improve only the evaluator. At each state, $Q^*(x, u) - J^*(x)$ is the extra cost of choosing u once and acting optimally afterward, rather than choosing an optimal action immediately. For finite state and control sets, the smallest such positive *action gap* is

$$\Delta = \min_{\substack{x \in \mathcal{X}, u \in \mathcal{U}(x) \\ Q^*(x, u) > J^*(x)}} [Q^*(x, u) - J^*(x)] > 0.$$

If there are no suboptimal actions, every policy is already optimal. Otherwise, replacing J^* by $\tilde{J}$ changes each one-step action score by at most $\alpha \|\tilde{J} - J^*\|_\infty$. Comparing two actions can therefore introduce at most twice this error. If $2\alpha \|\tilde{J} - J^*\|_\infty < \Delta$, the error is too small to make any suboptimal action beat an optimal action. For example, a true gap of 0.4 cannot be reversed when each score changes by at most 0.1. The greedy policy is then exactly optimal even

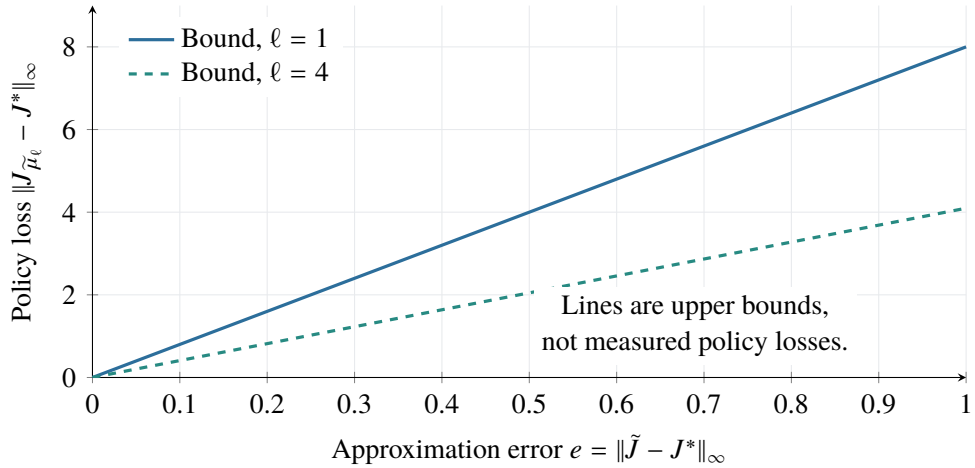


Figure 4.10: Global linear envelopes $2\alpha^\ell e / (1 - \alpha)$ with $\alpha = 0.8$. Increasing exact lookahead depth tightens the worst-case bound. An actual policy loss can lie well below its line. Concept corresponding to Bertsekas (2025, Fig. 1.5.13).

though the values are still approximate. For ℓ -step lookahead, the sufficient condition becomes $2\alpha^\ell \|\tilde{J} - J^*\|_\infty < \Delta$.

This finite-action argument explains why a nonzero value error can produce zero policy loss. In continuous-control problems there need not be a positive gap: nearby suboptimal controls may have arbitrarily similar costs. Nevertheless, a small change of control near a smooth optimum can have a much smaller effect on cost because its first-order variation vanishes. Figure 4.11 illustrates the related local behavior discussed in Bertsekas (2025, Fig. 1.5.14). Under appropriate regularity and closed-loop stability conditions, a local result can take the form $\|J_{\tilde{\mu}} - J^*\| = o(\|\tilde{J} - J^*\|)$. Here $o(\cdot)$ means that the ratio of policy loss to value error tends to zero as the evaluator approaches J^* . The policy loss shrinks faster than the value error, rather than following the straight global envelope. This is a local conclusion, not a consequence of discounting alone; far from the optimum, action choices can change substantially or a continuous system can become unstable. The LQ example below makes this contrast explicit, and Chapter 5 develops its geometric interpretation.

A scalar LQ illustration.

The numerical example behind Bertsekas (2025, Fig. 1.5.15) makes this difference visible without a full geometric construction. Consider $x_{k+1} = 2x_k + 2u_k$ with stage cost $x_k^2 + 5u_k^2$ and an undiscounted infinite horizon. Its optimal cost is $J^*(x) = 5x^2$. Using $\tilde{J}(x) = \tilde{K}x^2$ in one-step lookahead gives

$$\tilde{\mu}(x) = L_{\tilde{K}}x, \quad L_{\tilde{K}} = -\frac{4\tilde{K}}{5 + 4\tilde{K}}, \quad J_{\tilde{\mu}}(x) = K_{\tilde{\mu}}x^2, \quad K_{\tilde{\mu}} = \frac{1 + 5L_{\tilde{K}}^2}{1 - (2 + 2L_{\tilde{K}})^2}.$$

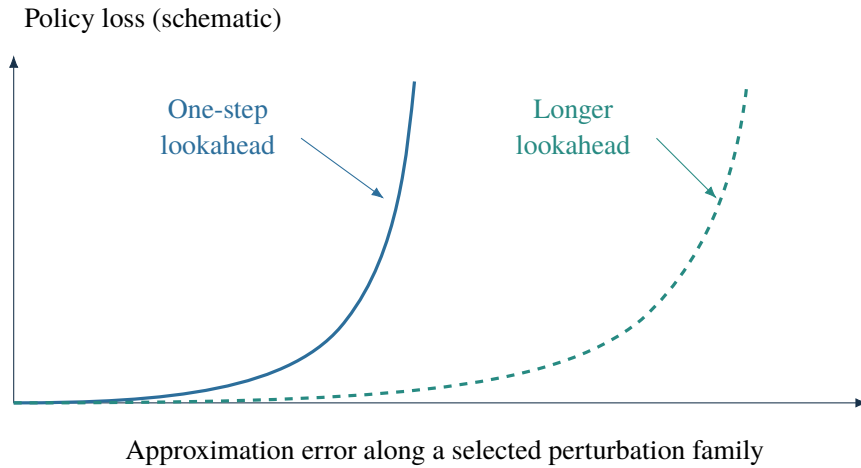


Figure 4.11: Local performance can be much flatter than a global linear bound; longer lookahead can extend the useful approximation region. Concept corresponding to Bertsekas (2025, Fig. 1.5.14). These curves are qualitative, not universal error bounds or numerical thresholds. Error magnitude alone does not determine policy loss: the direction of the approximation error also matters.

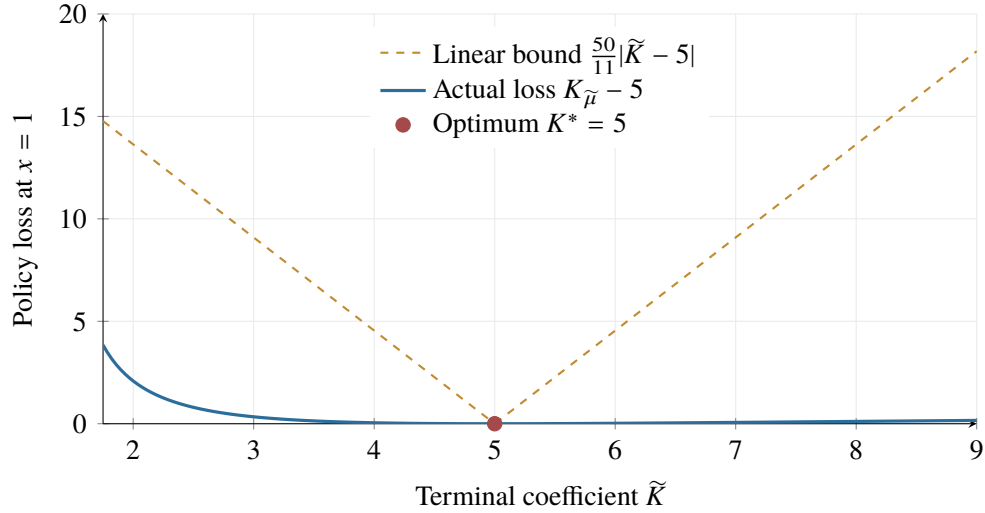


Figure 4.12: Recomputed scalar LQ example corresponding to Bertsekas (2025, Fig. 1.5.15 and Example 1.5.1): $a = b = 2$, $q = 1$, $r = 5$. The policy cost follows from the closed-loop geometric series. The comparison is in coefficient space on $\tilde{K} \geq 1.75$, with contraction modulus $\rho = 25/36$; the problem itself is undiscounted. The actual loss is small near the optimum and asymmetric on its two sides, unlike the linear bound.

The denominator comes from summing the closed-loop geometric series; it is positive for $\tilde{K} > 1.25$, the stability region considered here. At $x = 1$, the approximation error is $|\tilde{K} - 5|$ and the actual policy loss is $K_{\tilde{\mu}} - 5$.

This unbounded-state LQ problem lies outside assumption 4.1; its comparison uses quadratic *coefficients*, not a finite sup norm over all real states. The scalar Riccati map $F(K) = 1 + 20K/(5 + 4K)$ is a contraction on $[1.75, \infty)$ with modulus $\rho = F'(1.75) = 25/36$. The coefficient-level bound in the cited example is

$$K_{\tilde{\mu}} - 5 \leq \frac{2\rho}{1-\rho} |\tilde{K} - 5| = \frac{50}{11} |\tilde{K} - 5|, \quad \tilde{K} \geq 1.75.$$

Here ρ is a contraction modulus, not a discount factor. Figure 4.12 compares this bound with the actual loss computed from the displayed policy-evaluation formula. At $\tilde{K} = 3$ and 7 , both coefficient errors equal 2 , but the policy losses are approximately 0.339 and 0.065 , respectively; the bound is 9.091 in both cases. Near 5 , the actual loss is much flatter than the linear envelope, and its two sides are not symmetric.

The design message is therefore not merely to reduce a global fitting error. One can improve the evaluator, or spend online computation to transform it before selecting the current action. Exact lookahead contracts the error, while base-policy simulation offers a less expensive way to evaluate a plausible continuation. The next subsection combines these two uses of the online horizon; Chapter 5 develops their geometry.

4.7.1 Combining Multistep Lookahead, Rollout, and a Terminal Evaluator

Optimizing many future stages can be expensive because each stage introduces new candidate decisions. Following a base policy for part of that horizon avoids optimizing every future control, while a terminal evaluator avoids simulating forever. This is the practical compromise between pure lookahead and an entirely learned cost-to-go. The tree in Figure 4.13 separates three parts of an online calculation:

1. the first ℓ stages are optimized online;
2. the next m stages follow a declared base policy μ_{base} ; and
3. a learned or designed terminal evaluator $\hat{J}_T$ represents everything beyond those simulated stages.

The *root* is the current state x_k , at which the tree starts. The *root decision* is the first control selected there and applied to the system; the other decisions are predictions used to evaluate that choice. At the next observed state, the calculation starts again. Thus ℓ counts optimized stages including this first decision, whereas m counts subsequent base-policy stages. Neither is the number of evaluation updates made before an offline PI improvement.

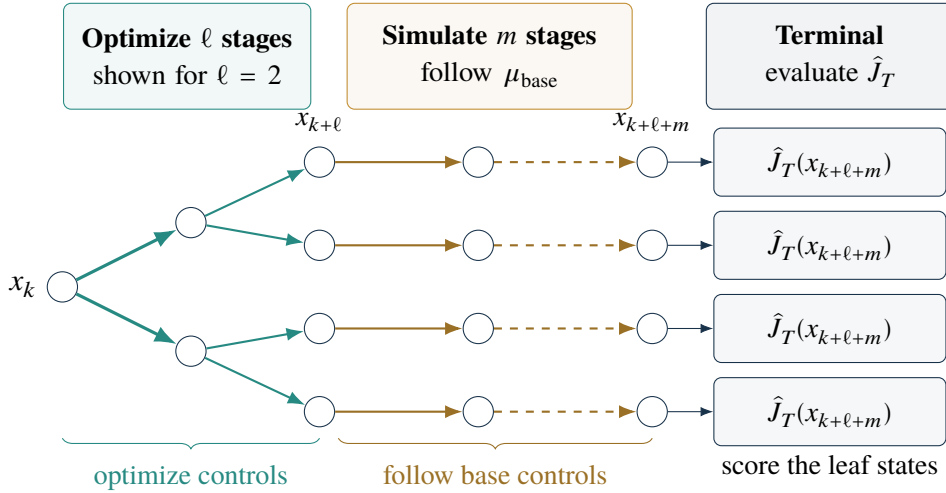


Figure 4.13: One online calculation combines optimized decisions, base-policy simulation, and terminal evaluation. The first branches at x_k are the root-action candidates; only the selected root action is executed. The diagram shows selected deterministic candidate paths with $\ell = 2$; stochastic transitions require additional outcome branches or repeated samples. Dashed segments allow an arbitrary base-policy depth m . Original rendering of the construction in Bertsekas (2019, Fig. 4.6.3).

From the terminal evaluator to the current decision.

To read the operator formula, recall that $TJ(x) = \min_{u \in \mathcal{U}(x)} \mathbb{E}[g(x, u, w) + \alpha J(f(x, u, w)) \mid x, u]$ optimizes one stage, whereas $T_\mu J(x) = \mathbb{E}[g(x, \mu(x), w) + \alpha J(f(x, \mu(x), w)) \mid x]$ fixes

that stage's control to $\mu(x)$. A power means repeated application, and a zeroth power is the identity. Starting at a successor x_{k+1} of the root action, $\ell - 1$ optimized stages remain before the m base-policy stages. Work backward from the terminal function to evaluate this remainder:

$$\bar{J}_{\ell,m} = T^{\ell-1} T_{\mu_{\text{base}}}^m \hat{J}_T, \quad (4.62)$$

The executed policy selects the current action using this evaluator:

$$\mu_{\ell,m}(x_k) \in \arg \min_{u \in \mathcal{U}(x_k)} \mathbb{E}[g(x_k, u, w) + \alpha \bar{J}_{\ell,m}(f(x_k, u, w)) \mid x_k, u]. \quad (4.63)$$

Its minimized objective, in contrast, includes that first stage:

$$\underbrace{(T\bar{J}_{\ell,m})(x_k)}_{\text{complete planning value}} = \min_{u \in \mathcal{U}(x_k)} \mathbb{E} \left[g(x_k, u, w) + \alpha \underbrace{\bar{J}_{\ell,m}(f(x_k, u, w))}_{\text{cost-to-go after the first action}} \mid x_k, u \right].$$

Hence $T_{\mu_{\ell,m}} \bar{J}_{\ell,m} = T\bar{J}_{\ell,m} = T^{\ell} T_{\mu_{\text{base}}}^m \hat{J}_T$. For example, with $\ell = 2$, the evaluator optimizes one further stage, and the root minimization supplies the other. For $\ell = 1$, no optimized stage remains inside $\bar{J}_{1,m}$: it is just the truncated base-policy cost-to-go. Thus the exponent $\ell - 1$ excludes the current decision; it does not omit a stage from the planning problem.

For a discounted policy operator, truncating the base-policy continuation has a transparent terminal-error relation:

$$\|T_{\mu_{\text{base}}}^m \hat{J}_T - J_{\mu_{\text{base}}}\|_{\infty} \leq \alpha^m \|\hat{J}_T - J_{\mu_{\text{base}}}\|_{\infty}. \quad (4.64)$$

Thus a longer base rollout reduces terminal error relative to the *base-policy value*, at additional simulation cost. Sampling variance also depends on the dynamics; it need not grow monotonically with depth. The optimal-value bound in Equation (4.61) concerns a different error:

$$\|T_{\mu_{\text{base}}}^m \hat{J}_T - J^*\|_{\infty} \leq \alpha^m \|\hat{J}_T - J_{\mu_{\text{base}}}\|_{\infty} + \|J_{\mu_{\text{base}}} - J^*\|_{\infty}.$$

Longer base-policy simulation reduces the first term, not the base policy's suboptimality. These statements assume exact operators; finite Monte Carlo samples, model mismatch, and pruned action sets contribute additional errors.

Table 4.4 compares the resulting designs. Its second column is the evaluator *after the root action*, not the complete planning value. In particular, an ℓ -step problem supplies $T^{\ell-1} \hat{J}_T$ to the root decision; optimizing the current action then produces $T^{\ell} \hat{J}_T$.

Proposition 4.5 (Lookahead with truncated base-policy simulation). *Under assumption 4.1, let $\ell, m \geq 1$, let μ be the base policy, and let $\hat{J}_T$ be bounded. The executed policy $\bar{\mu}$ is the root policy of ℓ optimized stages followed by m stages under μ and terminal cost $\hat{J}_T$. Then*

$$\|J_{\bar{\mu}} - J^*\|_{\infty} \leq \frac{2\alpha^{\ell}}{1-\alpha} \|T_{\mu}^m \hat{J}_T - J^*\|_{\infty}.$$

Table 4.4: Common online uses of a value-space approximation. The evaluator column excludes the root Bellman decision.

Variant	Evaluator after root action	What is optimized online	Main caution
One-step greedy	$\hat{J}_T$	current action only	state-value critic still requires successor prediction
ℓ-step lookahead	$T^{\ell-1}\hat{J}_T$	first ℓ actions/policies	branching and model error grow with depth
Pure rollout	$J_{\mu_{\text{base}}}$	current action against exact base value	exact infinite-horizon value is usually theoretical
Truncated rollout	$T_{\mu_{\text{base}}}^m \hat{J}_T$	current action; then m base steps	terminal bias and Monte Carlo variance must be reported
Combined search	$T^{\ell-1} T_{\mu_{\text{base}}}^m \hat{J}_T$	ℓ optimized stages; m base stages	computation, approximation, and search errors coexist

If additionally $T_{\mu} \hat{J}_T \leq \hat{J}_T + c$ at every state, then

$$J_{\bar{\mu}}(x) \leq \hat{J}_T(x) + \frac{c}{1-\alpha}, \quad x \in \mathcal{X}.$$

Proof. The first bound uses $T_{\mu}^m \hat{J}_T$ as the terminal function of the lookahead bound. For the second, put $H = \hat{J}_T + c/(1-\alpha)$, so $T_{\mu} H \leq H$. Repeated application of T_{μ} preserves this inequality, so $F = T_{\mu}^m H \leq H$ and $T_{\mu} F \leq F$. The minimizing operator can always choose μ , hence $TF \leq T_{\mu} F \leq F$. Applying the monotone operator T repeatedly now gives a decreasing sequence. Thus $V = T^{\ell-1} F$ satisfies $TV \leq V \leq F \leq H$. Constant shifts do not change the root policy, and $T_{\bar{\mu}} V = TV \leq V$. Repeated policy evaluation gives $J_{\bar{\mu}} \leq V \leq H$. $\square$

These are the two parts of Bertsekas (2019, Prop. 4.6.3). To interpret the second bound, suppose $\|\hat{J}_T - J_{\mu}\|_{\infty} \leq d$. Since $T_{\mu} J_{\mu} = J_{\mu}$, contraction gives, state by state,

$$T_{\mu} \hat{J}_T - \hat{J}_T = (T_{\mu} \hat{J}_T - T_{\mu} J_{\mu}) + (J_{\mu} - \hat{J}_T) \leq \alpha d + d.$$

Thus $c = (1+\alpha)d$ is valid. Combining the bound with $\hat{J}_T \leq J_{\mu} + d$ gives

$$J_{\bar{\mu}} \leq J_{\mu} + d + \frac{(1+\alpha)d}{1-\alpha} = J_{\mu} + \frac{2d}{1-\alpha}.$$

This is an approximate improvement certificate, not an assertion that arbitrary terminal fitting improves the base policy.

The connection to MPC and learned controllers.

Executing the first planned action and solving again at the next observed state is the receding-horizon structure used in MPC. This connection belongs to multistep lookahead generally, not just to rollout. A learned value can provide the terminal evaluator, and a learned policy can supply candidate controls or the base-policy continuation in Figure 4.13. Rollout is the particular construction that evaluates future costs under a base policy; it is not a synonym for all online planning. Chapter 5 develops the LQR/MPC connection, and Chapter 6 adds explicit learned policies.

4.7.2 Rollout as policy improvement

In exact PI, evaluating a policy μ_{base} gives $J_{\mu_{\text{base}}}$, and a greedy improvement with this function produces the next policy. Pure rollout performs that same improvement *at the current state during execution*, instead of first constructing and storing the improved policy at every state. Repeating the calculation at each visited state implements the improved feedback law. This is the PI connection; it does not require changing the base policy after every physical step.

Pure one-step rollout uses the base policy's exact value as the terminal evaluator:

$$\mu_{\text{ro}}(x) \in \arg \min_{u \in \mathcal{U}(x)} \mathbb{E}[g(x, u, w) + \alpha J_{\mu_{\text{base}}}(f(x, u, w)) \mid x, u]. \quad (4.65)$$

When the full feasible set is searched, the base action is among the candidates. The pure-rollout improvement result in proposition 2.7 therefore gives $J_{\mu_{\text{ro}}} \leq J_{\mu_{\text{base}}}$. If a restricted candidate set is used, it must retain the base action as a fallback. If $J_{\mu_{\text{base}}}$ is replaced by a finite Monte Carlo estimate or a learned critic, the exact monotonic guarantee is lost and Equation (4.35) becomes the relevant qualitative warning.

For reference, the next proposition restates proposition 2.7, including its restricted-search fallback condition. Here it identifies the exact guarantee that practical truncation and sampling must be compared against.

Proposition 4.6 (Rollout preserves or improves the base-policy cost). *Under assumption 4.1, let μ be an admissible base policy and let the candidate sets satisfy*

$$\mu(x) \in \overline{\mathcal{U}}(x) \subseteq \mathcal{U}(x), \quad x \in \mathcal{X}.$$

Suppose rollout attains the minimum of the exact one-step expectation with terminal function J_{μ} over $\overline{\mathcal{U}}(x)$ at every state. Then $J_{\mu_{\text{ro}}}(x) \leq J_{\mu}(x)$ for every x .

Proof. The feasible base action gives $T_{\mu_{\text{ro}}} J_{\mu} \leq T_{\mu} J_{\mu} = J_{\mu}$. Monotonicity and repeated policy evaluation yield $J_{\mu_{\text{ro}}} \leq J_{\mu}$. $\square$

This is Bertsekas (2019, Prop. 4.6.2). It requires neither an optimal base policy nor a search over every feasible control, but does require the exact comparison and inclusion of the base

action. The inequality is non-strict: an already optimal base policy, or a search that selects the base action everywhere, can give equality.

What if several base policies are available and each works well in a different part of the state space? Choosing one policy offline discards those complementary strengths. The following example shows how online rollout can combine them with a statewise guarantee: use the best of their exact costs at each successor state, while retaining their controls in the search.

Worked Example 4.5 (Rollout with several base policies). Let $\mu^1, \dots, \mu^M$ be base policies and set $H(x) = \min_{j=1, \dots, M} J_{\mu^j}(x)$. Thus H chooses the best base-policy cost separately at each state; it need not be the cost function of any one listed policy. At every state, retain all base-policy controls in the candidate set and minimize the one-step expected cost with terminal H . For every j ,

$$T_{\mu_{\text{ro}}} H \leq T_{\mu^j} H \leq T_{\mu^j} J_{\mu^j} = J_{\mu^j}.$$

Taking the minimum over j gives $T_{\mu_{\text{ro}}} H \leq H$, and consequently $J_{\mu_{\text{ro}}} \leq H \leq J_{\mu^j}$ for every j . The improved policy is no worse than any base policy, state by state.

For a numerical illustration, take two independent self-loop states, $\alpha = 0.5$, and two controls. Their costs are (1, 3) at state 1 and (3, 1) at state 2. The policies that always choose the first or always choose the second control have value vectors (2, 6) and (6, 2). Thus $H = (2, 2)$; rollout selects the first control at state 1 and the second at state 2, attaining (2, 2). For example, at state 1 its candidate scores are $1 + 0.5(2) = 2$ and $3 + 0.5(2) = 4$, so it selects the first control; the scores are reversed at state 2. Neither original policy is best everywhere, but the online choices combine their strengths. This illustrates the multiple-heuristic construction of Bertsekas (2019, Example 4.6.2). With estimated base values, the exact guarantee no longer follows automatically.

The exact $J_{\mu_{\text{base}}}$ in Equation (4.65) is an infinite-horizon mathematical object. It can be available analytically in a small finite-state, finite-control model or for a structured controller such as LQR, but one cannot generally obtain it by literally simulating an infinite trajectory. A practical rollout therefore reuses the truncated tail evaluator $\hat{J}_{\mu_{\text{base}}, m}$ in Equation (2.102), equivalently $T_{\mu_{\text{base}}}^m \hat{J}_T$. For each candidate $u \in \overline{\mathcal{U}}(x)$, it compares

$$\mathbb{E}[g(x, u, w) + \alpha \hat{J}_{\mu_{\text{base}}, m}(f(x, u, w)) \mid x, u].$$

This is the same root decision as pure rollout, with the exact base value replaced by the truncated tail. The candidate action comes first, followed by the m base-policy stages already defined in Chapter 2; no separate rollout convention is needed. For deterministic dynamics, one trajectory evaluates each candidate action. For stochastic dynamics, the expectation is normally estimated with multiple rollouts. Using common disturbance samples across candidate actions can reduce the variance of their *differences*, which matters because the controller selects an ordering rather than an absolute value (Bertsekas 2025, Secs. 2.7.3–2.7.4).

For a reproducible comparison, specify the candidate set, base policy, truncation depth, terminal evaluator, and simulation budget. These choices determine which practical approximation to pure rollout is being tested.

Table 4.5 summarizes where computation occurs for the controllers discussed in this section. A stored Q-function or actor mainly moves the control calculation offline; a value function used in lookahead leaves a model-based search to be performed at the current state. Thus the same offline evaluator can support different online controllers. Section 4.8.3 will compare those controllers on one fixed model, without retraining the evaluator as the lookahead depth changes.

Table 4.5: Offline and online roles of a value-space object.

Use	Offline requirement	Online requirement
Stored Q policy	Train $\tilde{Q}$ and validate action ordering	Evaluate all finite-action outputs and take arg min
One-step value greedy	Train $\tilde{J}$	Evaluate model successors and minimize one-step cost plus $\tilde{J}$
Multistep lookahead	Train terminal $\tilde{J}$ and possibly a base policy	Build or sample a tree, optimize root action, meet a time budget
Parameterized actor	Train or distill $\tilde{\mu}$	One forward pass, plus declared safety or constraint layer

4.8 Mobility Case: A Common Lane-Keeping Comparison

This section uses a compact stationary lane-keeping MDP related to the lateral control setting of the finite and periodic slalom study in Section 2.10, but it is not the same numerical model. The smaller nine-state model makes the learning-method comparisons easy to verify. Its state and control grids, disturbance distribution, stage cost, and discount factor are declared below and then held fixed while the representation and target mechanism change.

The normalized variables are

$$e \in \{-4, \dots, 4\}, \quad u \in \{-1, 0, 1\}, \quad \Pr(w = -1, 0, 1) = (0.10, 0.60, 0.30), \quad (4.66)$$

with

$$e_{k+1} = \text{clip}(e_k + u_k + w_k, -4, 4), \quad g(e, u) = e^2 + 0.20u^2, \quad \alpha = 0.95. \quad (4.67)$$

Here e is normalized lateral position error, u is a corrective control, and w is a disturbance. The clipping map saturates its argument at -4 and 4 ; it defines this finite-state teaching model, not physical constraint enforcement. The exact transition matrix is retained as a reference model. It is visible to exact and fitted VI, but it is not supplied to the MC, TD, or Q-learning updates. All final controller costs are evaluated with this same matrix, independently of the training procedure. The *mean policy cost* is $\bar{J}_\mu = \frac{1}{9} \sum_{e=-4}^4 J_\mu(e)$, the infinite-horizon cost averaged uniformly over the nine initial states. We separate three questions: what value approximation changes, how sampling affects evaluation and policy improvement, and what online search adds when the offline evaluator is fixed.

Six computations and their information requirements.

The first two computations provide exact-model references; the next four isolate representation error or sampled learning.

1. **Exact VI reference.** Synchronous model-based VI computes all nine values and all 27 state–action comparisons.
2. **Exact PI reference.** Starting from the zero-steering base policy, exact linear-system evaluation and exact model-based greedy improvement are alternated until the policy stops changing. This exposes the evaluation–improvement loop that exact VI hides inside every backup.
3. **Fitted VI.** The value is compressed to four coefficients,

$$\tilde{J}(e; r) = r_0 + r_1 z + r_2 z^2 + r_3 z^3, \quad z = e/4, \quad (4.68)$$

and each exact Bellman target is projected by ridge-regularized least squares over all nine states, with a cap of 400 sweeps and an iterate-change threshold of 10^{-10} . This isolates representation approximation; it does not reduce the number of model backups here.

4. **MC and TD(0) policy evaluation.** Both methods first evaluate the same myopic zero-steering policy from sampled transitions. Thirty seeds, uniformly randomized episode starts, 250 sampling episodes, and 100 transitions per episode are used. MC processes one return from each randomized episode start; TD(0) updates after every transition. These curves compare critics, not controllers.
5. **Approximate PI.** Alternate sampled MC or TD evaluation with exact-model greedy improvement for four rounds, using the same 250-episode budget per evaluation and the same 30 seeds. The first round uses the critics from computation 4; later rounds collect fresh data under the newly improved policies. Thus evaluation is model free, while improvement is model based.
6. **Tabular Q-learning.** The same 30 seeds each use 250 sampling episodes of 100 transitions, with uniformly randomized starts, decaying ϵ -greedy exploration, and pair-specific diminishing stepsizes. Q is updated after each transition, for 25,000 updates. There is no function approximation here: the approximation is asynchronous and sample based.

No DQN is trained on this nine-state problem. A neural Q-network would add optimization and hyperparameter effects where a 27-entry table is already an exact representation. The small experiment is used to verify the Bellman and sampling logic; DQN becomes meaningful when the state observation is too large or structured for a table.

This design makes two orthogonal axes visible. Fitted VI approximates the representation but uses complete model backups. Tabular Q-learning has an exact table representation for the finite MDP but approximates the Bellman expectation through samples. The MC/TD branch adds a third distinction: policy evaluation can be model free even when the subsequent J -based improvement is model based.

4.8.1 Exact DP and representation approximation

We first remove sampling from the comparison. Exact VI and PI solve the same finite-state problem; fitted VI uses the same model and Bellman targets but stores four coefficients instead of nine independent values. Figure 4.14 asks whether that compression changes the fitted values, the selected controls, or both.

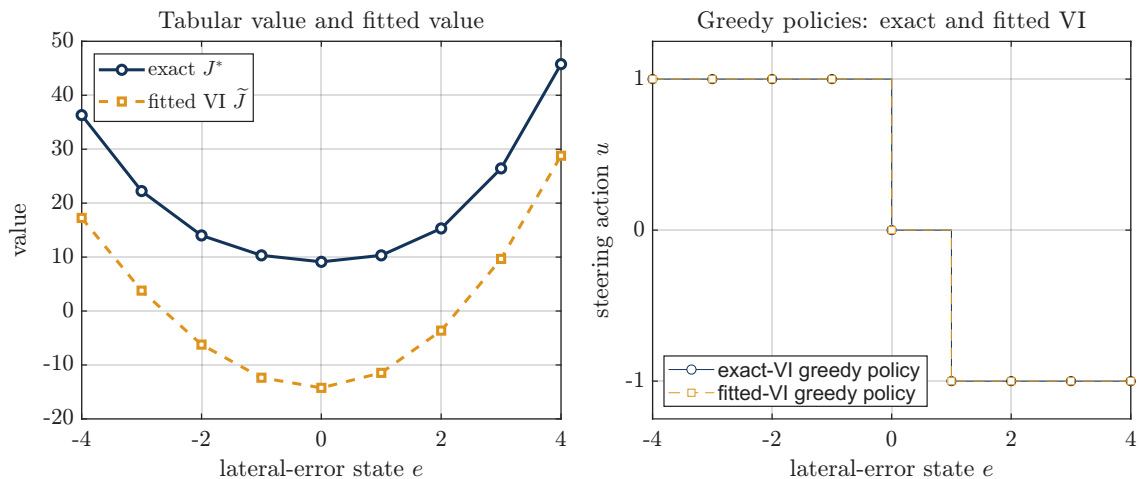


Figure 4.14: Representation approximation on the common lane-keeping model. Left: exact optimal cost-to-go and the four-coefficient fitted-VI result. Right: the state-to-control maps obtained by greedy minimization with the exact VI value and the fitted-VI value; their markers overlap at every state. Exact PI reaches the same final policy; its improvement history is shown in Figure 4.17.

The fitted value has RMSE 19.94 over the nine states, yet its greedy policy agrees with the exact policy at every state. Both controllers therefore have the same mean cost 21.097, averaged uniformly over initial states. This is the distinction from Chapter 3: the controller uses relative action costs, not the absolute accuracy of every fitted value. It is a result for this model, not a guarantee that large value errors are harmless.

Equal final policies do not mean equal computations.

Figure 4.15 separates numerical convergence from the final action comparison. A VI sweep updates all nine values and compares 27 state–control pairs. A fitted-VI sweep does the same model calculation and additionally solves a four-parameter least-squares problem. Using every state makes this a test of compression, not of the computational savings that fitted VI may offer when fitting from a subset of a large state space. Exact PI instead alternates a 9×9 policy-evaluation solve with 27 greedy comparisons. It reaches the optimal policy after four improvements; a further unchanged-policy check certifies termination. Exact VI uses 526 sweeps to reach an iterate change below 10^{-12} . The VI and fitted-VI diagnostics appear below; exact PI’s policy-cost history is compared with approximate PI in Figure 4.17.

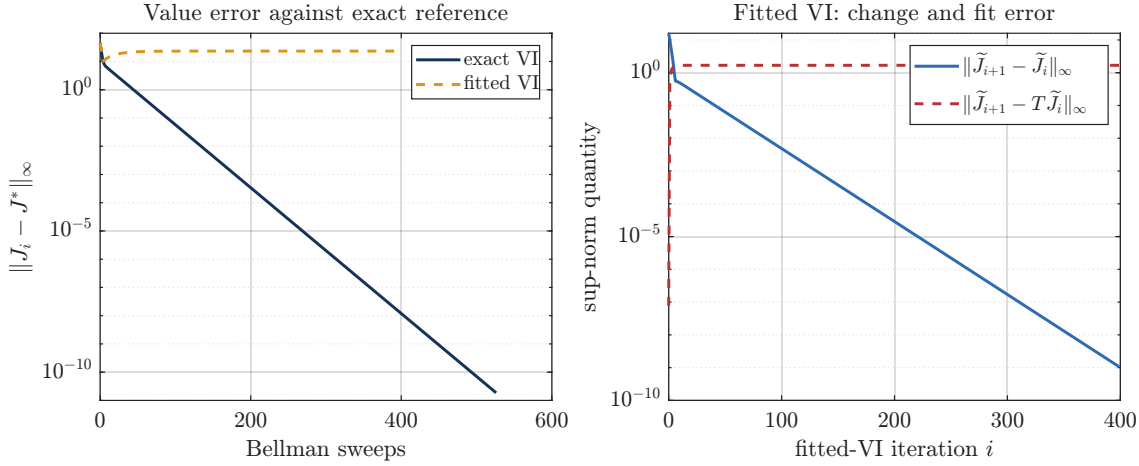


Figure 4.15: Iteration diagnostics without sampling. Left: sup-norm error relative to J^* for exact VI and fitted VI. Right: fitted VI’s *iterate change*, $\|\tilde{J}_{i+1} - \tilde{J}_i\|_\infty$, and *target-fit residual*, $\|\tilde{J}_{i+1} - T\tilde{J}_i\|_\infty$, which compares the fitted update with the Bellman target used to construct it. A small iterate change does not imply a small error relative to J^* .

Fitted VI reaches its 400-sweep cap with iterate change 1.06×10^{-9} , still above its 10^{-10} stopping threshold. Its target-fit residual remains 1.683 despite the small iterate change. Exact PI therefore needs few improvement rounds here, but each includes a policy-evaluation solve. The iteration counts alone do not determine which implementation runs faster.

4.8.2 Sampled evaluation and learned controllers

We now replace exact evaluation or expectation with sampled data. Each comparison below keeps its question separate: first evaluate one fixed policy, then improve policies repeatedly, and finally learn an optimal Q-factor directly.

How much data is being used?

The budgets in the six computations are experimental choices, not standard requirements for MC, TD, or Q-learning. A common budget is $250 \times 100 = 25,000$ sampled transitions: one fixed-policy evaluation, one approximate-PI evaluation round, or one Q-learning run. Four PI rounds therefore use 100,000 transitions, not the same total as one Q-learning run. All sampled comparisons use 30 independent runs with matching seed IDs; their data are not pooled to train one controller. Matching seeds and budgets does not make trajectories identical when the policies differ.

In this implementation, MC retains only the return from each randomized episode start: 250 return targets per evaluation. TD uses all 25,000 transitions for scalar updates. MC could reuse additional visited states, but that is not the estimator tested here. The curves therefore compare these declared implementations, not an intrinsic sample-efficiency ranking of all MC and TD

methods.

The sampling episodes are 100-step segments of a continuing task, not episodes ending in a terminal state. MC uses $\sum_{k=0}^{99} \alpha^k g_k$ with *zero terminal cost*, so the remaining discounted tail is omitted. TD(0) instead uses $g_k + \alpha \hat{J}(e_{k+1})$, with $\hat{J}$ its current estimate, at every step, including the last step of a segment; it does not force that successor estimate to zero. Q-learning likewise bootstraps at the boundary. Uniform random starts provide coverage through a resettable simulator and need not be available in physical operation.

First question: how accurately is one policy evaluated?

Figure 4.16 holds the zero-steering base policy fixed. Its exact cost-to-go is the reference for MC and TD; J^* is not the reference in this comparison. The left panel shows the learned functions, and the right panel measures evaluation RMSE as more transitions become available. The bands show variability across independent training runs.

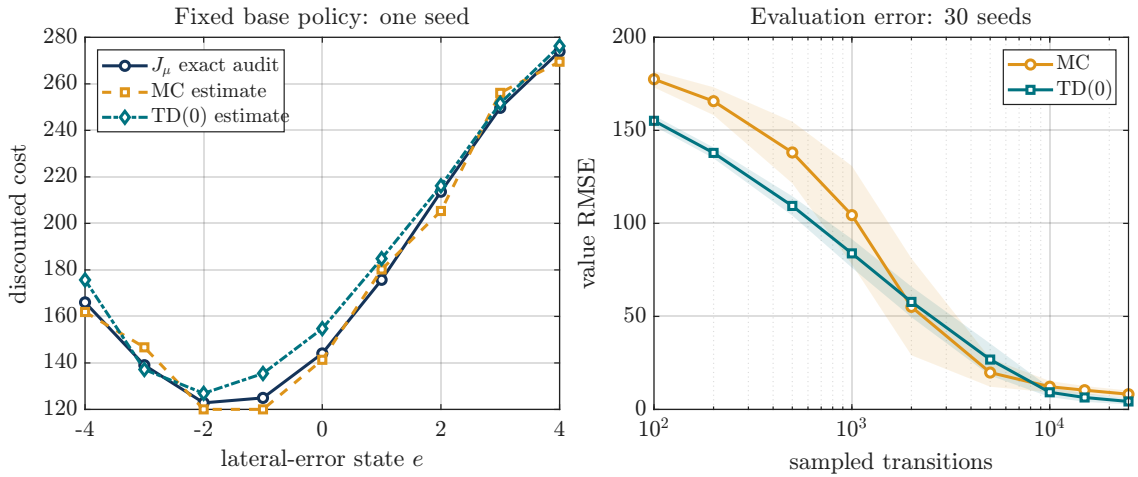


Figure 4.16: MC and TD(0) evaluate the same fixed base policy. Evaluation error is measured against $J_{\mu_{\text{base}}}$, not J^* . Left: the first declared seed. Right: learning curves with the same transition budget; shaded bands show one standard deviation across 30 seeds, clipped at zero for nonnegative error. MC uses episode-start returns, whereas TD updates at every transition.

At the final budget, mean evaluation RMSE is 8.19 for MC and 4.30 for TD. Neither number is the cost of an improved controller. It measures only how accurately that method has evaluated the unchanged base policy, whose mean cost is 178.884.

Second question: what changes after policy improvement?

The critics just evaluated now drive the first exact-model greedy improvement. This is round 1 of approximate PI, not an additional improvement before that comparison. Subsequent rounds evaluate the new policy and improve it again. In Figure 4.17, exact PI and both approximate-PI methods start from the same base policy. The horizontal axis counts policy improvements,

not training samples. The sampled methods use 25,000 transitions per round; exact PI uses model-based linear-system evaluation. Their common vertical axis is the mean infinite-horizon policy cost $\bar{J}_\mu$ defined above.

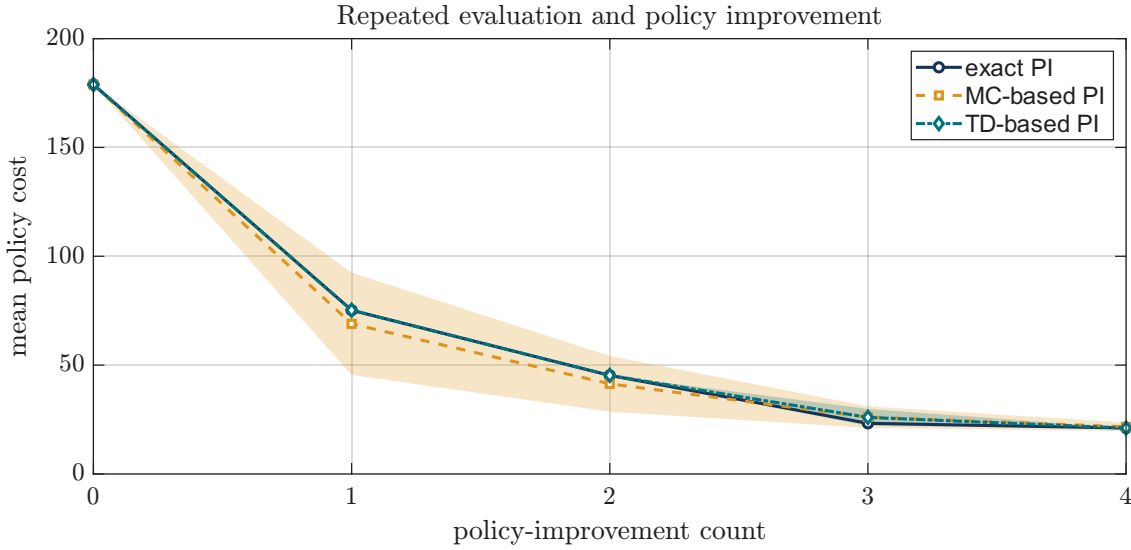


Figure 4.17: Controller cost after successive policy improvements. Exact PI and MC-/TD-based approximate PI start from the same base policy. Sampled evaluations use 250 length-100 episodes per round; means and one-standard-deviation bands are over 30 seeds. Each point evaluates the resulting stationary policy, not its fitted critic.

Exact PI's first improved policy costs 75.225, well above the eventual optimum 21.097. A similarly high one-round approximate-PI cost is therefore not, by itself, evidence of a poor critic. After four rounds, TD-based approximate PI reaches the exact policy in all 30 runs, while MC-based approximate PI has mean cost 21.658 and mean action agreement 98.1%. The remaining differences reflect finite-data evaluation and truncation, not different control objectives. The shaded bands make their variability visible.

Third question: when does Q-learning produce a good controller?

Q-learning does not evaluate that fixed base policy. It changes its Q table using sampled optimality targets while exploring. Figure 4.18 therefore reports Q error and greedy-policy cost separately, both against the available exact reference. The Q error concerns all 27 numerical entries; the cost depends on which of three controls is selected at each of nine states. The right panel's *mean policy cost gap* is

$$\bar{J}_\mu - \bar{J}^* = \frac{1}{9} \sum_{e=-4}^4 [J_\mu(e) - J^*(e)],$$

where μ is greedy with respect to the current Q table and $\bar{J}^*$ is the mean optimal cost. Both costs are evaluated with the exact model; the gap is not an error in the learned Q values.

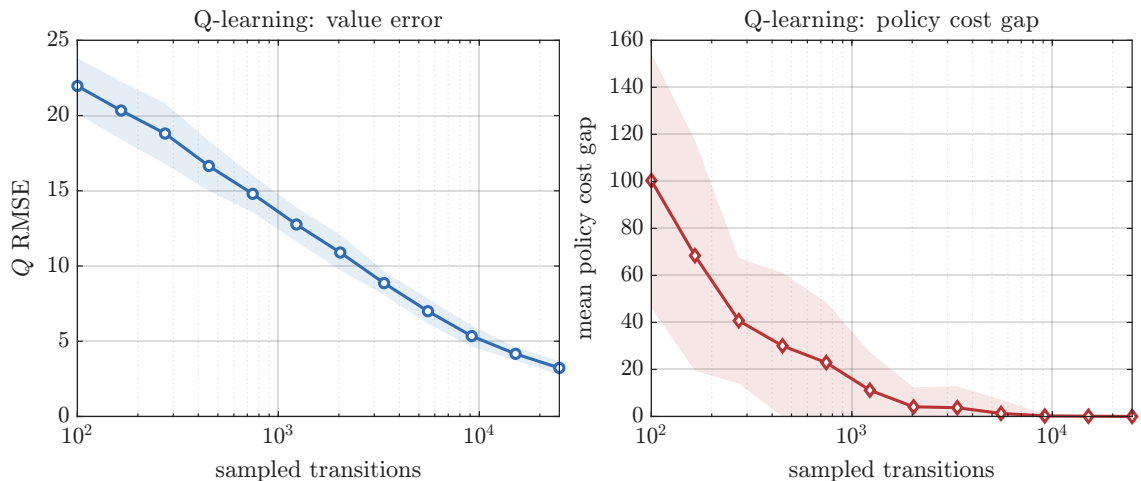


Figure 4.18: Q-learning accuracy and control performance versus sampled transitions. Left: Q-table RMSE relative to Q^* . Right: mean policy cost gap $\bar{J}_\mu - \bar{J}^*$, averaged uniformly over initial states and computed by exact policy evaluation. Means and one-standard-deviation bands are over 30 seeds; the bands are clipped at zero for nonnegative error and cost gap. Numerical Q error and controller cost gap are distinct diagnostics.

After 25,000 transitions, every run selects an optimal action at all nine states and has a numerically zero policy cost gap, even though Q RMSE remains nonzero. Thus the action ordering can become correct before the whole Q table becomes accurate. This finite-budget observation is not a general guarantee that 25,000 samples suffice for Q-learning.

A common controller-level summary.

Table 4.6 gathers the final policy costs without mixing them with critic errors. Related methods share a row group. Both the one- and four-improvement MC/TD rows come from the preceding 30-seed experiment with 250 episodes per round. This holds the per-round data budget fixed when comparing how repeated policy improvement changes performance.

Some sampled candidates cost less than the corresponding exact-PI iterate: sampling error can change an action comparison in a direction that happens to help. Exact PI greedily improves the current policy; it does not promise the globally best policy after each intermediate iteration. These occasional gains therefore do not establish that inaccurate evaluation is preferable. Equal final costs do not imply equal data use, storage, or computation: fitted VI stores four parameters, tabular VI/PI stores nine values, and Q-learning stores 27 Q entries, with the different update requirements specified above.

Table 4.6: Controller performance evaluated with the common exact model. Costs are averaged uniformly over the nine initial states and, for sampled methods, over all declared seeds.

Method	Evaluation point	Action agreement	Mean policy cost
Exact VI	Final	100%	21.097
Exact PI	One improvement	66.7%	75.225
	Four improvements	100%	21.097
Fitted VI	400 sweeps	100%	21.097
MC-based approximate PI	One improvement	68.5%	68.971
	Four improvements	98.1%	21.658
TD-based approximate PI	One improvement	66.7%	75.225
	Four improvements	100%	21.097
Q-learning	25,000 transitions	100%	21.097
Base policy	No improvement	11.1%	178.884

4.8.3 Online improvement with a fixed offline evaluator

The preceding experiments improve a stored policy through repeated offline learning or DP. We now isolate the question from Section 4.7: can more online computation improve control without another training round? Keep the same model and the same zero-steering base policy. Freeze its exact cost-to-go $J_{\mu_{\text{base}}}$, and vary only the lookahead depth ℓ :

$$\mu_{\ell}(e) \in \arg \min_{u \in \{-1, 0, 1\}} \mathbb{E}[g(e, u) + \alpha (T^{\ell-1} J_{\mu_{\text{base}}})(e^+) | e, u].$$

At $\ell = 1$, this is pure rollout: the first action is optimized, and the base policy supplies its continuation cost. It selects the same policy as the first exact-PI improvement, not the converged PI policy. Larger ℓ optimizes more future decisions while retaining the same terminal cost-to-go. No evaluator parameters are updated.

To include representation error as well, fit a second terminal evaluator $\hat{J}_{\text{base}}(e) = b_0 + b_1 z + b_2 z^2$, $z = e/4$, by ordinary least squares to the same nine base-policy values. Its RMSE is 10.03. It is also frozen before varying ℓ ; this is a controlled compression test, not a sample-based learning experiment.

Figure 4.19 also tests truncated rollout. For that comparison, $\ell = 1$ is fixed: the root action is followed by m stages of the unchanged base policy and then a zero terminal estimate. Its tail is $T_{\mu_{\text{base}}}^m 0$, rather than the exact $J_{\mu_{\text{base}}}$. Varying m improves the approximation of base-policy continuation; it does not add m freely optimized stages. This distinction is precisely why lookahead depth and rollout length should not be treated as the same horizon.

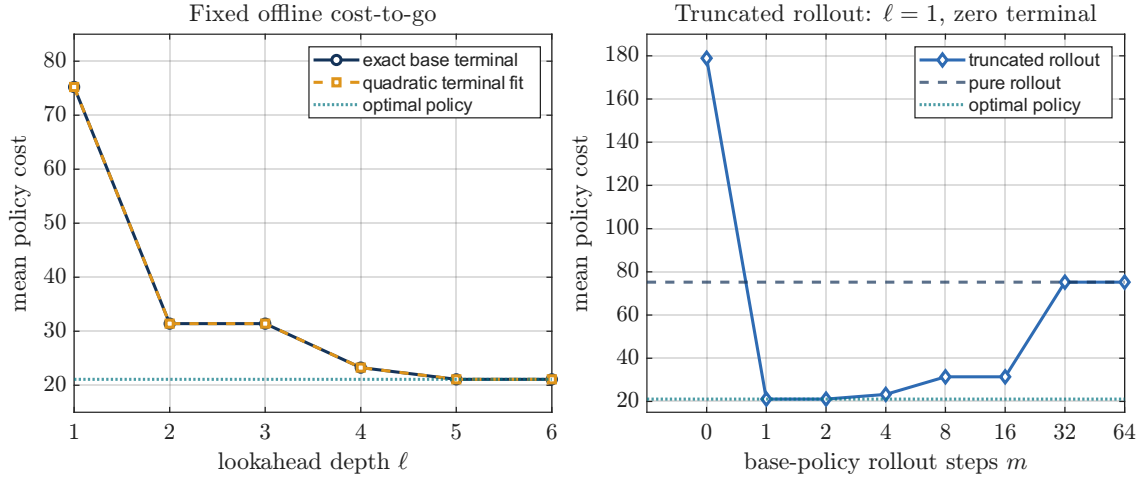


Figure 4.19: Online improvement on the same lane-keeping model. Left: lookahead with either the exact base value or its frozen quadratic fit; the curves coincide. Right: truncated rollout with $\ell = 1$ and zero terminal cost. Lookahead varies optimized stages; rollout varies base-policy stages m . Every reported cost is the infinite-horizon cost of the repeatedly executed root policy, not the finite planning objective. Exact expectations remove sampling variation from this comparison.

Pure rollout's mean cost is 75.225, already below the base cost 178.884. With either frozen evaluator, lookahead costs at $\ell = 2, 3, 4, 5$ are 31.411, 31.411, 23.269, and 21.097, respectively. Thus five optimized stages recover the optimal policy here, without further learning.

The truncated-rollout result is not monotone: $m = 1$ and $m = 2$ already select the optimal policy, whereas $m = 32$ and $m = 64$ select pure rollout's more costly policy. A longer base-policy tail estimates $J_{\mu_{\text{base}}}$ more accurately, but this need not make its greedy policy closer to the optimum. The short zero-terminal approximation happens to select better controls in this model. This does not contradict the pure rollout guarantee, which compares with the base policy, not with every truncated approximation or with the optimum. The comparison illustrates the distinct roles of terminal information, base-policy simulation, and optimized prediction in Section 4.7. It is an illustrative finite-model test, not an empirical proof of the general error bounds.

The improvement requires more computation for each control decision. Count one model calculation as evaluating a state–control pair across its three possible disturbances. Averaged over the nine possible current states, the search uses 3 such calculations at $\ell = 1$ and 88.3 at $\ell = 5$ (at most 99). Repeated state–remaining-depth subproblems are computed once within each search. These counts measure online action selection, excluding offline fitting and the later calculation of policy costs for the graphs. A stored policy instead needs only a lookup.

4.9 A Linear-Programming View of Value Approximation

VI and PI construct the optimal cost-to-go through repeated Bellman calculations. An alternative is to solve for all its state values together as the decision variables of an LP. This section

uses a finite-state, finite-control discounted problem: let $\mathcal{X}$ contain n states, write $\bar{g}(x, u) = \mathbb{E}[g(x, u, w) \mid x, u]$, and let $p(x' \mid x, u)$ denote the transition probabilities. The model data and $0 < \alpha < 1$ are fixed.

Why maximizing lower bounds gives the exact value.

Consider any candidate vector $J = (J(x))_{x \in \mathcal{X}}$ satisfying

$$J(x) \leq \bar{g}(x, u) + \alpha \sum_{x' \in \mathcal{X}} p(x' \mid x, u) J(x'), \quad x \in \mathcal{X}, \quad u \in \mathcal{U}(x). \quad (4.69)$$

Requiring the inequality for every admissible control is equivalent to $J \leq TJ$. Monotonicity gives $J \leq TJ \leq T^2J \leq \dots$, and discounted VI converges to J^* ; hence every feasible J satisfies $J \leq J^*$ componentwise.

The other half of the argument is that J^* itself is feasible. Its Bellman equation selects the smallest right-hand side at each state, so it satisfies every inequality in Equation (4.69). Thus J^* is not merely an upper limit on feasible candidates: it belongs to the feasible set and is its componentwise largest vector. For any weights $c(x) > 0$, it follows that

$$\max_J \sum_{x \in \mathcal{X}} c(x) J(x) \quad (4.70)$$

$$\text{subject to } J(x) \leq \bar{g}(x, u) + \alpha \sum_{x' \in \mathcal{X}} p(x' \mid x, u) J(x'), \quad x \in \mathcal{X}, \quad u \in \mathcal{U}(x) \quad (4.71)$$

has the unique solution $J = J^*$. Indeed, any different feasible vector is strictly smaller at at least one state and no larger anywhere else; strict positivity of $c(x)$ then makes its objective strictly smaller. The LP is linear because its transition probabilities and costs are fixed coefficients, while the state values are its unknowns (Bertsekas 2019, Sec. 4.10).

There is no conflict with the cost-minimization convention. We are not maximizing a controller's incurred cost. We are maximizing a certified *lower bound* on the minimum achievable cost, and the largest such bound is the minimum cost itself.

Worked Example 4.6 (A two-state Bellman-inequality LP). Let $\alpha = 0.5$ and give each state two deterministic controls. At state 1, staying costs 2 and moving to state 2 costs 1. At state 2, staying costs 4 and moving to state 1 costs 7. Substitution in Equation (4.69) gives

$$\begin{aligned} \text{state 1, stay: } & J(1) \leq 2 + 0.5J(1), \quad \text{or } J(1) \leq 4, \\ \text{state 1, move: } & J(1) \leq 1 + 0.5J(2), \\ \text{state 2, stay: } & J(2) \leq 4 + 0.5J(2), \quad \text{or } J(2) \leq 8, \\ \text{state 2, move: } & J(2) \leq 7 + 0.5J(1). \end{aligned}$$

The vector $(4, 8)$ satisfies all four constraints and the Bellman equations: $4 = \min\{4, 5\}$ and $8 = \min\{8, 9\}$. It is therefore J^* . No feasible point can have a first coordinate above 4 or a second coordinate above 8, so maximizing $J(1) + J(2)$ selects this point.

Figure 4.20 shows all four boundaries and their common feasible region. Moving a constant-objective line toward larger $J(1) + J(2)$ reaches its last feasible contact at J^* . Positive unequal

weights tilt these lines but select the same point, since J^* dominates every feasible vector componentwise.

Fewer value parameters, but still many constraints.

The exact LP has n unknown state values and $M = \sum_{x \in \mathcal{X}} |\mathcal{U}(x)|$ constraints. For a large state space, even storing and solving this problem can be impractical without special structure. We therefore introduce Chapter 3's linear architecture $\tilde{J}(x; r) = \phi(x)^\top r$, where $r \in \mathbb{R}^d$ and $d \ll n$. Replacing the free value vector by this representation gives

$$\max_r \sum_{x \in \mathcal{X}} c(x) \phi(x)^\top r, \quad (4.72)$$

$$\text{subject to } \phi(x)^\top r \leq \bar{g}(x, u) + \alpha \sum_{x' \in \mathcal{X}} p(x' | x, u) \phi(x')^\top r, \quad x \in \mathcal{X}, u \in \mathcal{U}(x). \quad (4.73)$$

This is an *approximate LP*: it remains linear in its d parameters, but need not represent J^* exactly. With all constraints enforced, every feasible represented value is still a lower bound on J^* . Unlike in the exact LP, changing the weights can now change the best represented lower bound, because the architecture couples values at different states.

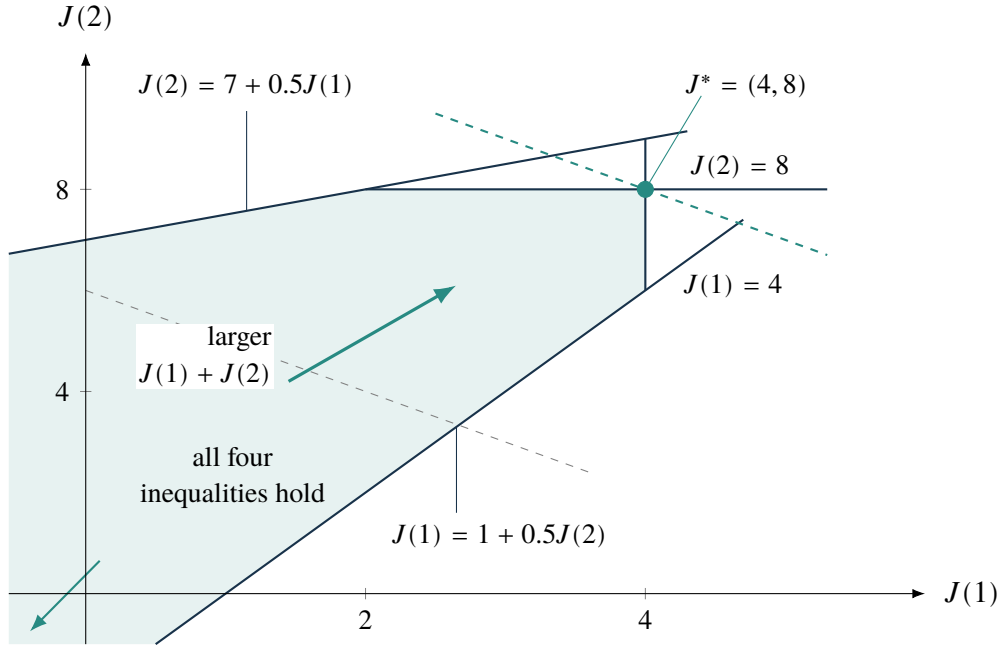


Figure 4.20: Four Bellman inequalities in the two-state example. Each solid line is an equality boundary; the shaded intersection contains the feasible cost vectors and continues toward the lower left beyond the displayed window. Dashed lines are constant values of $J(1) + J(2)$, not additional constraints. The largest feasible vector J^* is also the unique maximizer for every strictly positive weighting of the two coordinates. Original discounted example illustrating the geometry of Bertsekas (2019, Fig. 4.10.1).

The main computational obstacle has not disappeared: there are still M constraints, even if d is small. With a controls at each of n states, there are na inequalities. For example, $n = 10^6$, $a = 10$, and $d = 100$ give only 100 unknown parameters but ten million constraints. Evaluating, storing, and checking those constraints may dominate the calculation. Low-dimensional value representation is therefore not, by itself, a tractable solution method (Bertsekas 2019, Sec. 4.10).

Reducing constraints is a separate approximation.

A practical method enforces only a selected set $\tilde{\mathcal{C}} \subseteq \{(x, u) : x \in \mathcal{X}, u \in \mathcal{U}(x)\}$. One can sample initial constraints, solve the smaller LP, then search for violated inequalities and add their state–control pairs. The objective may likewise use a selected set of relevant states. However, removing constraints enlarges the feasible set: a value feasible on the sample need not be a lower bound at every state, and a poorly chosen subset can even leave the objective unbounded. No violations on a finite audit set is not a global certificate. Reliable constraint selection and violation search remain substantive parts of the method, not automatic consequences of choosing linear features.

The same idea can evaluate a fixed policy by retaining only the constraint for $u = \mu(x)$ at each state, reducing the full constraint count from M to n . It can thus supply a policy-evaluation component of approximate PI. The main contrast with fitted VI remains useful: fitted VI fits Bellman targets, whereas approximate LP optimizes a value representation subject to one-sided Bellman inequalities. Both require attention to where the approximation is accurate, but they enforce that approximation differently.

4.10 Chapter Summary

Chapter 3 supplied function representations and fitting methods. This chapter specified what those functions should learn and how their outputs become control decisions. The two main routes are approximate VI, which targets Bellman optimality, and approximate PI, which separates evaluation of a current policy from its improvement. Table 4.7 summarizes their implementations and the online uses of the resulting cost-to-go functions.

- **Fitted VI turns Bellman targets into compact evaluators.** Function fitting represents the updated cost-to-go with a chosen architecture. Bounds on successive fitting errors relate this approximation to value and policy accuracy; convergence of the fitted update alone does not imply recovery of J^* .
- **Sampled targets support complementary learning methods.** Q-learning targets optimality, whereas MC and TD evaluate a stated policy. SARSA couples on-policy Q-factor evaluation with control. DQN adds replay and a target network; Double DQN changes target construction, and Dueling DQN changes the Q representation.
- **Approximate PI separates evaluation from improvement.** Returns, TD updates, or linear evaluators supply the current-policy cost-to-go, and optimistic PI adjusts evaluation effort before the next improvement. Evaluation and action-comparison errors enter separately into the resulting performance bounds.

Table 4.7: Value-space methods: learned object, target, and control decision.

Method	Learned object	Target or constraint	Control decision
Fitted VI	$\tilde{J}$ or $\tilde{Q}$	Bellman optimality backup, then fitting	Greedy minimization
Q-learning / DQN	$\tilde{Q}$ targeting Q^*	Sampled optimality target	Finite-action arg min
Approximate PI	$\tilde{J}_\mu$ or $\tilde{Q}_\mu$	Current-policy evaluation	Separate improvement
MC / TD	$\tilde{J}_\mu$ or $\tilde{Q}_\mu$	Return or bootstrapped policy target	None without improvement
SARSA	$\tilde{Q}_\mu$ for a changing policy	Sampled next action under the behavior policy	On-policy control update
Lookahead / rollout	Fixed evaluator or base-policy cost-to-go	Predicted costs and terminal evaluation	Apply optimized root action
Approximate LP	$\phi(x)^\top r$	Bellman inequalities	Greedy minimization

- **Offline value approximation supports online improvement.** Multistep lookahead optimizes a prefix; rollout uses a base policy to evaluate continuation. Exact rollout is no worse than its base under the stated conditions. Optimizing more stages and estimating the base-policy tail more accurately serve different purposes.
- **A common control problem makes the tradeoffs visible.** In the lane-keeping example, fitted VI has substantial value error but selects the exact policy. Sampling, repeated PI, and deeper online search offer different ways to improve control. Reporting value error, policy cost, sample budget, and planning work identifies their respective contributions.
- **Bellman inequalities offer an optimization-based route.** The exact LP recovers J^* by maximizing feasible lower bounds; linear features give a compact approximate LP. Its state–control constraints still require attention, and reducing them introduces a separate approximation.

The central control connection is the combination of offline value approximation and online policy improvement. Chapter 5 develops this relation through LQR, MPC, and geometric interpretation. Chapter 6 then makes the policy itself an approximation object and places PPO, TD3, and SAC within the evaluation–improvement framework established here.

Bibliography

- Bertsekas, D. P. (2019). *Reinforcement Learning and Optimal Control*. Belmont, MA: Athena Scientific. ISBN: 9781886529397.
- (2025). *A Course in Reinforcement Learning*. 2nd ed. Nashua, NH: Athena Scientific. ISBN: 9781886529298.
- Hasselt, H. van, A. Guez, and D. Silver (2016). “Deep Reinforcement Learning with Double Q-Learning”. In: *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pp. 2094–2100. DOI: [10.1609/aaai.v30i1.10295](https://doi.org/10.1609/aaai.v30i1.10295). URL: <https://ojs.aaai.org/index.php/AAAI/article/view/10295>.
- Mnih, V., K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis (2015). “Human-Level Control Through Deep Reinforcement Learning”. In: *Nature* 518.7540, pp. 529–533. DOI: [10.1038/nature14236](https://doi.org/10.1038/nature14236). URL: <https://doi.org/10.1038/nature14236>.
- Singh, S., T. Jaakkola, M. L. Littman, and C. Szepesvári (2000). “Convergence Results for Single-Step On-Policy Reinforcement-Learning Algorithms”. In: *Machine Learning* 38.3, pp. 287–308. DOI: [10.1023/A:1007678930559](https://doi.org/10.1023/A:1007678930559).
- Sutton, R. S. and A. G. Barto (2018). *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press. URL: <http://incompleteideas.net/book/the-book-2nd.html>.
- Wang, Z., T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas (2016). “Dueling Network Architectures for Deep Reinforcement Learning”. In: *Proceedings of the 33rd International Conference on Machine Learning*. Vol. 48. Proceedings of Machine Learning Research, pp. 1995–2003. URL: <https://proceedings.mlr.press/v48/wangf16.html>.
- Watkins, C. J. C. H. and P. Dayan (1992). “Q-Learning”. In: *Machine Learning* 8.3–4, pp. 279–292. DOI: [10.1007/BF00992698](https://doi.org/10.1007/BF00992698). URL: <https://doi.org/10.1007/BF00992698>.